

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University

Accredited by NAAC with "B++" grade



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PCCSL408 DATABASE MANAGEMENT SYSTEMS LAB

LAB MANUAL

PREPARED BY:

Ms. ANITHA L

APPROVED BY:

HOD, CSE

Vision of the Institution

Progress through education

Mission of the Institution

To seek, strive for and scale greater heights of quality education

M1: Seek to foster in every student a profound and practical understanding of knowledge, encouraging critical inquiry and intellectual curiosity.

M2: Strive to ensure every student develops holistically, cultivating the skills, ethical character, and resilience necessary to thrive in diverse and challenging future environments.

M3: Scale the commitment to service by inspiring students to apply their knowledge and skills to actively address community needs, promoting ethical leadership and social responsibility.

Vision of the Department

Progress through quality education and become a centre of excellence in computing knowledge

Mission of the Department

M1: To seek and strive for advancements in technological domains.

M2: To create socially responsible engineers in the field of computer science

M3: To bridge the gap between science and mankind through meaningful innovation and service

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program: B. Tech	Department: Computer Science and Engineering
Course: Dbms Lab	Semester: S4 Batch: A Credits: 2
Course Code: PCCSL408	Regulation: 2024
Course Area/Domain: General Engineering	L: 0 T: 0 TW/SL: 1 P: 3
Corresponding Lab Course Code (If Any): Nil	Lab Course Name: Nil

COURSE PRE-REQUISITES:

CODE	COURSE NAME	DESCRIPTION	SEM
PCCST402	Database Management Systems	A sound knowledge of the basics of relational DBMS	S4

COURSE OUTCOMES:

SL NO	DESCRIPTION
CO1	Develop database schema for a given real world problem-domain using standard design and modeling approaches
CO2	Construct queries using SQL for database creation, interaction, modification, and update.
CO3	Plan and implement cursors, procedures, functions, and control structures using PL/SQL
CO4	Perform CRUD operations in NoSQL Databases.
CO5	Design database applications using front-end tools and back-end DBMS

CO - PO/PSO MAPPING:

CO Number	PO STRENGTH											PSO STRENGTH	
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	3	3	3	1								2	2
CO2	3	3	3	1								2	2
CO3	3	3	3	1								2	2
CO4	3	3	3	2	3							2	2
CO5	3	3	3	2	3							3	3
Total	3	3	3	1.4	1.2							2.2	2.2

COURSE SYLLABUS**LAB Experiments:**

1. Design a database schema for an application with an ER diagram from a problem description.
2. Creation of database schema - DDL (create tables, set constraints, enforce relationships, create indices, delete and modify tables). Export ER diagram from the database and verify relationships (with the ER diagram designed in step 1).
3. Database initialization - Data insert, Data import to a database (bulk import using UI and SQL Commands).
4. Practice SQL commands for DML (insertion, updating, altering, deletion of data, and viewing/querying records based on condition in databases).
5. Implementation of various aggregate functions, Order By, Group By & Having clause in SQL.
6. Implementation of set operators nested queries, and join queries.
7. Practice of SQL TCL DCL commands like Rollback, Commit, Savepoint, Practice of SQL DCL commands for granting and revoking user privileges.
8. Practice of SQL commands for creation of views and assertions.
9. Creation of Procedures, Triggers and Functions.

10. Creation of Packages and cursors
11. Design a database application using any front-end tool for any problem selected in experiment number
 - 1.The application constructed should have five or more tables
12. Perform basic CRUD (Create, Read, Update, Delete) operations on a Cassandra table.
13. Write and execute CQL queries to retrieve specific data from Cassandra tables
14. Create a simple application using Mongoddb with python.

GENERAL INSTRUCTIONS (for the lab)**Preparation:**

- **Software Installation:** Ensure you have the required DBMS software installed (e.g., MySQL, PostgreSQL, PLSQL, Oracle, or SQL Server). Download and install the appropriate community or student edition.
- **Client Tools:** Install a suitable client or IDE for interacting with the database server (e.g., MySQL Workbench, pgAdmin, SQL Developer, or the DBMS vendor's command-line interface).
- **Familiarization:** Review the basic SQL syntax for Data Definition Language (DDL) and Data Manipulation Language (DML).

Lab Setup:

- **Directory Creation:** Create a new directory for the DBMS lab session (e.g., DBMS_Lab_S1_CSE_A) to store all your .sql script files and reports.
- **Database Access:** Ensure you have the necessary credentials (username and password) and connection details (hostname, port) to access the database server.
- **Server Check:** Ensure an Oracle Database instance (like Oracle XE) is running on your Windows machine.
- **IDE:** Use Oracle SQL Developer or a similar IDE. Ensure you have a working connection to your Oracle schema.
- **Debugging Tool:** Within your SQL Developer worksheet, always run SET SERVEROUTPUT ON; to enable output from DBMS_OUTPUT.PUT_LINE, which is crucial for testing PL/SQL logic.

Documentation:

- **Script Header:** Start each SQL script file with comments (--) that include your name, date, the Experiment/Program Number, and a brief description of the script's purpose.
- **Inline Comments:** Use comments to explain the purpose of complex queries, stored procedures, triggers, or any part of the database design (like schema creation or integrity constraints).

Coding Standards:

- SQL Standards: While there is no official PEP 8 equivalent, follow consistent SQL coding standards:
 - Use uppercase for SQL keywords (e.g., SELECT, FROM, WHERE).
 - =Use meaningful and descriptive names for tables, columns, and other database objects.
 - Use a consistent casing style (e.g., snake_case or PascalCase) for identifiers.
- Indentation and Formatting: Properly indent your SQL queries, especially for multi-line commands like CREATE TABLE, JOIN queries, or PL/SQL blocks, to improve readability.

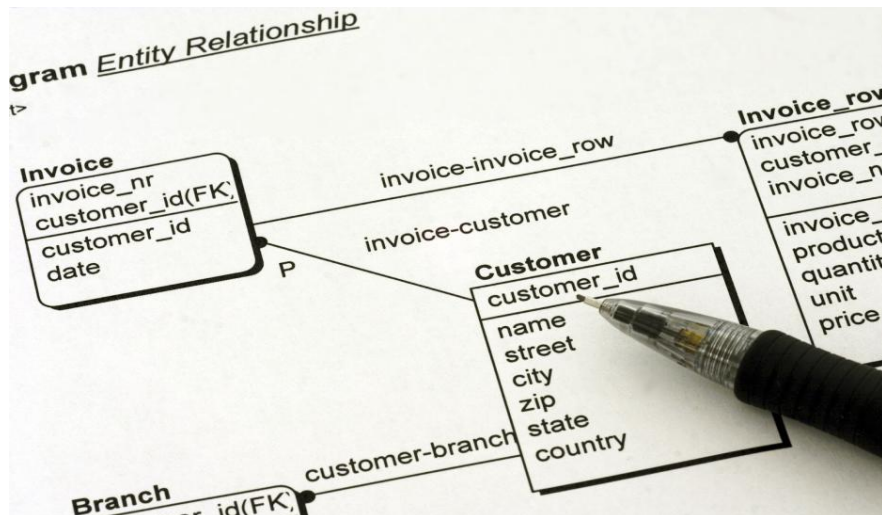
Testing and Debugging:

- Verify Schema: After using Data Definition Language (DDL) commands (CREATE, ALTER, DROP), always use commands like DESCRIBE <table_name> or its equivalent to verify the structure.
- Test Data Integrity: After using Data Manipulation Language (DML) commands (INSERT, UPDATE, DELETE), use a SELECT query to immediately verify the data has been modified as expected.
- Transaction Control: Practice using Transaction Control Language (TCL) commands like COMMIT and ROLLBACK to test operations safely before permanently applying changes.

BEST PRACTICES

Understand the Basics:

- Conceptual Clarity: Ensure a solid understanding of fundamental concepts like Database Architecture,
E-R Modeling



Relational Algebra, and Normalization (1NF, 2NF, 3NF).

- **SQL Command Types:** Differentiate clearly between DDL (Data Definition), DML (Data Manipulation), DCL (Data Control), and TCL (Transaction Control) commands. Plan Your Database Design:
- **E-R Diagram First:** Before writing any CREATE TABLE statements, design the database structure using an Entity-Relationship (E-R) diagram.
- **Normalization:** Apply appropriate normalization techniques to eliminate data redundancy and anomalies before implementing the schema.

Write Readable SQL:

- **Integrity Constraints:** Always define Primary Keys, Foreign Keys, NOT NULL, UNIQUE, and CHECK constraints when creating tables to enforce data integrity at the database level.
- **Efficient Queries:** Learn to write optimized SQL queries. Use JOIN operations (Inner, Outer, Self) correctly instead of multiple simple queries where a join is more efficient.
- **Use Variables/PL-SQL:** For complex logic, practice using Stored Procedures, Functions, and Triggers in the procedural language (like PL/SQL or T-SQL) provided by your RDBMS.

Data Management:

- **Backup and Restore:** Understand and practice basic database backup and restore operations, even if it's just for your lab environment.
- **Security (DCL):** Understand the use of Data Control Language (DCL) commands like GRANT and REVOKE to manage user permissions and access control.

Practice and Resources:

- **Manual/Problem Sets:** Always refer to the specific lab manual and the problem statement before starting an experiment.
- **Online Documentation:** Utilize the official documentation for your RDBMS (e.g., Oracle or MySQL documentation) as your primary reference for syntax and features.

INDEX

Exp No.	Expt. Name	Page Number
1	University Management System Er Diagram	20
2	Implementation Of Ddl Commands	23
3	Data Insertion And Bulk Import Using Dml Commands And Ui Tools	28
4	Sql Dml: Essential Data Manipulation Operations	34
5	Implementation Of Sql Aggregate Functions	40
6	Implementation Of Set Operators, Nested Queries, And Join Queries	46
7 A	Implementation Of Sql Tcl Commands	54
7 B	Implementation Of Sql Dcl Commands	60
8	Creation And Manipulation Of Tables, Views, And Assertions In Sql	65
9 A	Creation And Execution Of Stored Procedures	70
9 B	Creation And Implementation Of Database Triggers	73
9 C	Creation And Execution Of User-Defined Functions	77
10 A	Creation And Implementation Of Pl/Sql Packages	80
10 B	Implementation Of Cursors In Pl/Sql	83
11	Database Application Design Using A Front-End Tool	87
12	Crud Operations On Cassandra Database	99
13	Retrieving Specific Data From Cassandra Using Cql	104
14	Simple Crud Application Using MongoDB And Python	108

INTRODUCTION TO SQL

History of SQL

Dr. E. F. Codd published the paper, "A Relational Model of Data for Large Shared Data Banks", in June 1970 in the Association of Computer Machinery (ACM) journal, Communications of the ACM. Codd's model is now accepted as the definitive model for relational database management systems (RDBMS). The language, Structured English Query Language ("SEQUEL") was developed by IBM Corporation, Inc., to use Codd's model. SEQUEL later became SQL (still pronounced "sequel"). In 1979, Relational Software, Inc. (now Oracle Corporation) introduced the first commercially available implementation of SQL. Today, SQL is accepted as the standard RDBMS language.

How SQL Works

The strengths of SQL provide benefits for all types of users, including application programmers, database administrators, managers, and end users. Technically speaking, SQL is a data sub-language. The purpose of SQL is to provide an interface to a relational database such as Oracle, and all SQL statements are instructions to the database. SQL differs from general-purpose programming languages like C and BASIC. Among the features of SQL are the following:

1. It processes sets of data as groups rather than as individual units.
2. It provides automatic navigation to the data.
3. It uses statements that are complex and powerful individually, and that therefore stand alone.

Flow-control statements were not part of SQL originally, but they are found in the recently accepted optional part of SQL, ISO/IEC 9075-5: 1996. Flow-control statements are commonly known as "persistent stored modules" (PSM), and Oracle's PL/SQL extension to SQL is similar to PSM.

Essentially, SQL lets you work with data at the logical level. You need to be concerned with the implementation details only when you want to manipulate the data. For example, to retrieve a set of rows from a table, you define a condition used to filter the rows. All rows satisfying the condition are retrieved in a single step and can be passed as a unit to the user, to another SQL statement, or to an application. You need not deal with the rows one by one, nor do you have to worry about how they are physically stored or retrieved.

All SQL statements use the optimizer, a part of Oracle that determines the most efficient means of accessing the specified data. Oracle also provides techniques you can use to make the optimizer perform its job better.

SQL provides statements for a variety of tasks, including:

1. Querying data

2. Inserting, updating, and deleting rows in a table
3. Creating, replacing, altering, and dropping objects
4. Controlling access to the database and its objects
5. Guaranteeing database consistency and integrity

SQL unifies all of the above tasks in one consistent language.

Common Language for All Relational Databases

All major relational database management systems support SQL, so you can transfer all skills you have gained with SQL from one database to another. In addition, all programs written in SQL are portable. They can often be moved from one database to another with very little modification.

Summary of SQL Statements

SQL statements are divided into these categories:

1. Data Definition Language (DDL) Statements
2. Data Manipulation Language (DML) Statements
3. Transaction Control Statements (TCL)
4. Session Control Statement
5. System Control Statement

Managing Tables

A table is a data structure that holds data in a relational database. A table is composed of rows and columns. A table can represent a single entity that you want to track within your system. This type of a table could represent a list of the employees within your organization, or the orders placed for your company's products.

A table can also represent a relationship between two entities. This type of a table could portray the association between employees and their job skills, or the relationship of products to orders. Within the tables, foreign keys are used to represent relationships.

Creating Tables

To create a table, use the SQL command `CREATETABLE`.

Syntax:

CREATE TABLE <TABLE NAME>(<FIELD NAME ><DATA TYPE><[SIZE]>,,.....)

Altering Tables

Alter a table in an Oracle database for any of the following reasons:

1. To add one or more new columns to the table
2. To add one or more integrity constraints to a table
3. To modify an existing column's definition (datatype, length, default value, and NOTNULL integrity constraint)
4. To modify data block space usage parameters (PCTFREE, PCTUSED)
5. To modify transaction entry settings (INITTRANS, MAXTRANS)
6. To modify storage parameters (NEXT, PCTINCREASE, etc.)
7. To enable or disable integrity constraints associated with the table
8. To drop integrity constraints associated with the table

When altering the column definitions of a table, you can only increase the length of an existing column, unless the table has no records. You can also decrease the length of a column in an empty table. For columns of data type CHAR, increasing the length of a column might be a time consuming operation that requires substantial additional storage, especially if the table contains many rows. This is because the CHAR value in each row must be blank-padded to satisfy the new column length.

If you change the datatype (for example, from VARCHAR2 to CHAR), then the data in the column does not change. However, the length of new CHAR columns might change, due to blank-padding requirements.

Altering a table has the following implications:

1. If a new column is added to a table, then the column is initially null. You can add a column with a NOT NULL constraint to a table only if the table does not contain any rows.
2. If a view or PL/SQL program unit depends on a base table, then the alteration of the base table might affect the dependent object, and always invalidates the dependent object.

Privileges Required to Alter a Table

To alter a table, the table must be contained in your schema, or you must have either the ALTER object privilege for the table or the ALTER ANY TABLE system privilege.

Dropping Tables

Use the SQL command DROP TABLE to drop a table. For example, the following statement drops the

EMP_TAB table:

If the table that you are dropping contains any primary or unique keys referenced by foreign keys to other tables, and if you intend to drop the FOREIGN KEY constraints of the child tables, then include the CASCADE option in the DROP TABLE command.

Oracle Built-In Datatypes

A datatype associates a fixed set of properties with the values that can be used in a column of a table or in an argument of a procedure or function. These properties cause Oracle to treat values of one datatype differently from values of another datatype. For example, Oracle can add values of sNUMBER datatype, but not values of RAW datatype.

Oracle supplies the following built-in data types: character data types

- CHAR
- NCHAR
- VARCHAR2 and VARC
- NVARCHAR2
- CLOB
- NCLOB
- LONG
 - NUMBER datatype
 - DATE datatype
 - Binary datatypes
- BLOB
- BFILE
- RAW
- LONG RAW

Another datatype, ROWID, is used for values in the ROWID pseudocolumn, which represents the unique address of each row in a table.

Table summarizes the information about each Oracle built-in datatype.

Summary of Oracle Built-In Data types

Using Character Data types

Use the character data types to store alphanumeric data.

1. CHAR and NCHAR data types store fixed-length character strings.

2. VARCHAR2 and NVARCHAR2 data types store variable-length character strings. (The VARCHAR datatype is synonymous with the VARCHAR2 datatype.)
3. CLOB and NCLOB data types store single-byte and multi-byte character strings of up to four gigabytes.
4. The LONG datatype stores variable-length character strings containing up to two gigabytes, but with many restrictions.
5. This data type is provided for backward compatibility with existing applications; in general, new applications should use CLOB and NCLOB data types to store large amounts of character data.

When deciding which datatype to use for a column that will store alphanumeric data in a table, consider the following points of distinction:

Space Usage

1. To store data more efficiently, use the VARCHAR2 datatype. The CHAR datatype blank-pads and stores trailing blanks up to a fixed column length for all column values, while the VARCHAR2 datatype does not blank-pad or store trailing blanks for column values.
2. Use the CHAR data type when you require ANSI compatibility in comparison semantics (when trailing blanks are not important in string comparisons). Use the VARCHAR2 when trailing blanks are important in string comparisons.

Comparison Semantics

Use the CHAR data type when you require ANSI compatibility in comparison semantics (when trailing blanks are not important in string comparisons). Use the VARCHAR2 when trailing blanks are important in string comparisons.

Future Compatibility

1. The CHAR and VARCHAR2 data types are and will always be fully supported. At this time, the VARCHAR datatype automatically corresponds to the VARCHAR2 datatype and is reserved for future use.
2. CHAR, VARCHAR2, and LONG data is automatically converted from the database character set to the character set defined for the user session by the NLS_LANGUAGE parameter, where these are different.

Using the NUMBER Datatype

Use the NUMBER datatype to store real numbers in a fixed-point or floating-point format. Numbers using this data type are guaranteed to be portable among different Oracle platforms, and offer up to 38

decimal digits of precision. You can store positive and negative numbers of magnitude 1×10^{-130} to 9.99×10^{125} , as well as zero, in a NUMBER column.

For numeric columns you can specify the column as a floating-point number:

Column_name NUMBER

Or, you can specify a precision (total number of digits) and scale (number of digits to the right of the decimal point):

Column_name NUMBER (<precision>, <scale>)

Although not required, specifying the precision and scale for numeric fields provides extra integrity checking on input. If a precision is not specified, then the column stores values as given. The table shows examples of how data would be stored using different scale factors.

Using the DATE Datatype

Use the DATE datatype to store point-in-time values (dates and times) in a table. The DATE datatype stores the century, year, month, day, hours, minutes, and seconds.

Oracle uses its own internal format to store dates. Date data is stored in fixed-length fields of seven bytes each, corresponding to century, year, month, day, hour, minute, and second.

Date Format

For input and output of dates, the standard Oracle default date format is DD-MON-YY.

For example: '13-NOV-92'

To change this default date format on an instance-wide basis, use the NLS_DATE_FORMAT parameter. To change the format during a session, use the ALTER SESSION statement. To enter dates that are not in the current default date format, use the TO_DATE function with a format mask.

For example:

TO_DATE ('November 13, 1992', 'MONTH DD, YYYY')

If the date format DD-MON-YY is used, then YY indicates the year in the 20th century (for example, 31-DEC-92 is December 31, 1992). If you want to indicate years in any century other than the 20th century, then use a different format mask, as shown above.

Time Format

Time is stored in 24-hour format #HH:MM:SS. By default, the time in a date field is 12:00:00 A.M. (midnight) if no time portion is entered. In a time-only entry, the date portion defaults to the first day of the current month. To enter the time portion of a date, use the TO_DATE function with a format mask indicating the time portion, as in:

```
INSERT INTO Birthdays_tab (bname, bday) VALUES ('ANNIE',TO_DATE('13-NOV-92 10:56 A.M.','DD-MON-YY HH:MI A.M.'));
```

To compare dates that have time data, use the SQL function TRUNC if you want to ignore the time component.

Use the SQL function SYSDATE to return the system date and time. The FIXED_DATE initialization parameter allows you to set SYSDATE to a constant; this can be useful for testing.

INTRODUCTION TO ER MODEL

The ER model represents real world situations using concepts, which are commonly used by people. It allows defining a representation of the real world at a logical level. ER model has no facilities to describe machine-related aspects.

In the ER model the logical structure of data is captured by indicating the grouping of data into entities. The ER model also supports a top-down approach by which details can be given in successive stages.

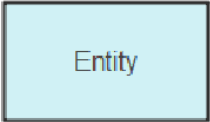
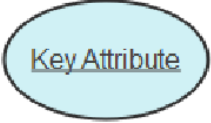
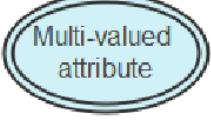
Entity: An entity is something which is described in the database by storing its data, it may be a concrete entity or a conceptual entity.

Entity set: An entity set is a collection of similar entities.

Attribute: An attribute describes a property associated with entities. Attributes will have a name and a value for each entity.

Domain: A domain defines a set of permitted values for an attribute

SYMBOLS IN E-R DIAGRAM

	Entity		Attribute
	Weak Entity		Key Attribute
	Associative Entity		Key Attribute
	Relationship		Key Attribute
	Identifying Relationship		Derived Attribute
	Mandatory Relationship		Optional Relationship
	Partial Participation		Total Participation

EXP NO:1**UNIVERSITY MANAGEMENT SYSTEM ER DIAGRAM****AIM**

A university registrar's office maintains data about the following entities: (a) courses, including number, title, credits, syllabus, and prerequisites; (b) course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom; (c) students, including student-id, name, and program; and (d) instructors, including identification number, name, department, and title. Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled. Construct an E-R diagram for the registrar's office. Document all assumptions that you make about the mapping constraints.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Analyze a problem description and extract necessary entities and attributes.
2. Identify appropriate relationships and cardinalities among entities.
3. Construct a clear, normalized ER diagram based on requirements.
4. Develop a database schema from the ER diagram.
5. Recognize the importance of keys (primary, foreign) in relational design.
6. Apply conceptual modeling techniques to design real-world databases.
7. Document assumptions when real-world data is incomplete or ambiguous.
8. Translate conceptual models into relational models ready for implementation.

DESCRIPTION

Designing a database schema from a problem description is a fundamental step in database development. The process begins with understanding the real-world scenario, identifying important objects(entities),determining their properties (attributes), and analyzing how these entities interact (relationships).To represent this visually, an Entity–Relationship (ER) diagram is created.

An ER diagram typically includes:

- Entities: Objects like Student, Course, Instructor, etc.
- Attributes: Properties like student_id, course_title, instructor_name.
- Primary keys: Unique identifiers for each entity.
- Relationships: How entities are connected (e.g., Student *enrolls* in Course).
- Cardinality: 1:1, 1:N, or M:N constraints between entities.

After designing the ER diagram, it is mapped to a logical relational schema, which is later implemented in a database system (SQL).

This process ensures:

- Data consistency
- Avoidance of redundancy
- Clear structure for storage and retrieval
- Correct modeling of real-world requirements

OUTPUT

The main entity sets are student, course, course-offering, and instructor. The entity set course-offering is a weak entity set dependent on course. The assumptions made are :

- A class meets only at one particular place and time. This E-R diagram cannot model a class meeting at different places at different times.
- There is no guarantee that the database does not have two classes meeting at the same place and time

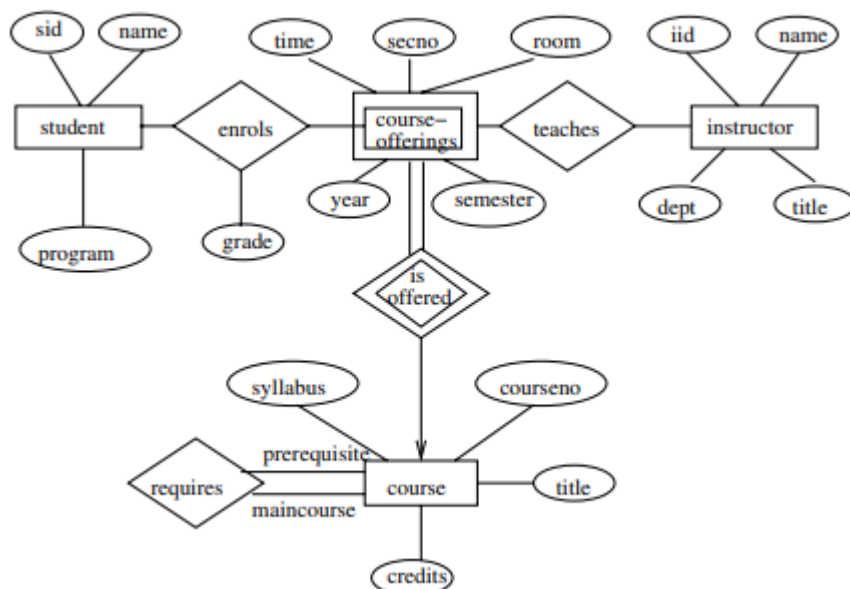


Fig. ER Diagram for a University

MODEL VIVA QUESTIONS

1. What is an ER diagram?
2. Define an entity in the context of database design.
3. What is an attribute? Give examples from your design
4. What is the purpose of a primary key?
5. What is a foreign key and why is it used?

RESULT

ER diagram has been drawn successfully. By constructing an ER diagram, I was able to apply a standard design and modelling approach

EXP NO: 2**IMPLEMENTATION OF DDL COMMANDS****AIM**

Creation, modification, configuration, and deletion of databases Commands

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Write SQL DDL commands to create, modify, and delete database tables.
2. Define and apply constraints such as primary key, foreign key, unique, check, and not null.
3. Establish referential integrity between tables using foreign keys.
4. Create and manage indices to optimize database performance.
5. Alter table structures by adding, deleting, or modifying columns and constraints.
6. Drop database objects safely and understand the consequences.
Understand the role of DDL in database design and implementation.
7. Translate ER diagrams into actual database schemas using SQL

DESCRIPTION

Data Definition Language (DDL) is a subset of SQL used to define and manage the structure of a database. It is responsible for creating, modifying, and deleting database objects such as tables, constraints, views, and indexes.

DDL commands allow database designers to translate a conceptual or logical design into an actual physical database. They help ensure that the database structure follows integrity rules and maintains consistency.

Key Components of DDL**1. CREATE**

Used to create database objects such as tables, schemas, views, and indexes.

Syntax: CREATE TABLE <relation_name/table_name >

(field_1 data_type(size),field_2 data_type(size), ..);

2. ALTER

Used to modify an existing database object.

ALTER TABLE ...ADD...: This is used to add some extra fields into existing relations.

Syntax: ALTER TABLE relation_name ADD (new field_1 data_type(size), new field_2 data_type(size),...);

ALTER TABLE...MODIFY...: This is used to change the width as well as data type of fields of existing relations.

Syntax: ALTER TABLE relation_name MODIFY (field_1 newdata_type(Size), field_2 newdata_type(Size), ... field_newdata_type(Size));

ALTER TABLE..DROP ...: This is used to remove any field of existing relations.

Syntax: ALTER TABLE relation_name DROP COLUMN (field_name);

ALTER TABLE..RENAME...: This is used to change the name of fields in existing relations.

Syntax: ALTER TABLE relation_name RENAME COLUMN (OLD field_name) to (NEW field_name);

3. DROP

DROP TABLE: This is used to delete the structure of a relation. It permanently deletes the records in the table.

Syntax: DROP TABLE relation_name;

Example: SQL>DROP TABLE std

CONSTRAINTS

Used to enforce data integrity. Common constraints include:

1. PRIMARY KEY
2. FOREIGN KEY
3. UNIQUE

4. CHECK

5. NOT NULL

PRIMARY KEY -A primary key uniquely identifies each row in a table. No two rows can have the same value, and it cannot be NULL.

FOREIGN KEY - A foreign key links one table to another.It ensures the value must already exist in the referenced (parent) table.

UNIQUE - Ensures all values in a column are different.No duplicates allowed, but NULL is allowed (unless NOT NULL is added).

CHECK - Ensures values meet a specific condition.

Example: age must be greater than 18.

NOT NULL - Prevents a column from having empty (NULL) values.A value must be entered.

REFERENTIAL INTEGRITY - Ensures relationships between tables are maintained using foreign keys.

INDEXES - Improve the speed of data retrieval but may increase storage overhead.

Through DDL operations, designers ensure the database is structured, efficient, and consistent with application requirements.

QUERY QUESTIONS

1. Write an SQL query to create tables STUDENT and COURSE with appropriate datatypes.
2. Write SQL queries to create Table With Foreign Keys (Enrollments)
3. Write an SQL command to modify a table by adding a new column (e.g., email in STUDENT table make email mandatory).
4. Write an SQL query to add a new column to COURSE table
5. Write a query to create an INDEX on frequently searched columns (e.g., student_name or course_title).
6. Write an SQL command to drop/delete a table (e.g., DROP the ENROLLMENT table).
7. Write the steps (or SQL support queries where applicable) to export the ER diagram from the database using DBMS tools (MySQL Workbench).

8. Write steps to verify relationships in the exported ER diagram and compare with the ER diagram designed in Step 1.

SQL COMMANDS

1. Create Table: Students

```
CREATE TABLE Students ( student_id INT PRIMARY KEY, full_name VARCHAR(100) NOT NULL,email VARCHAR(100) UNIQUE, dob DATE );
```

2. Create Table: Courses

```
CREATE TABLE Courses (course_id INT PRIMARY KEY, course_name VARCHAR(100) NOT NULL,credits INT CHECK (credits > 0));
```

3. Create Table With Foreign Keys (Enrollments)

```
CREATE TABLE Enrollments (enrollment_id INT PRIMARY KEY, student_id INT NOT NULL,course_id INT NOT NULL,enrollment_date DATE DEFAULT CURRENT_DATE, CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES Students(student_id), CONSTRAINT fk_course FOREIGN KEY (course_id) REFERENCES Courses(course_id));
```

4. Add a Constraint (Make email mandatory)

```
ALTER TABLE Students MODIFY email VARCHAR(100) NOT NULL;
```

5. Add a New Column to a Table

```
ALTER TABLE Courses ADD description TEXT;
```

6. Create an Index (Improves search on student name)

```
CREATE INDEX idx_student_name ON Students(full_name);
```

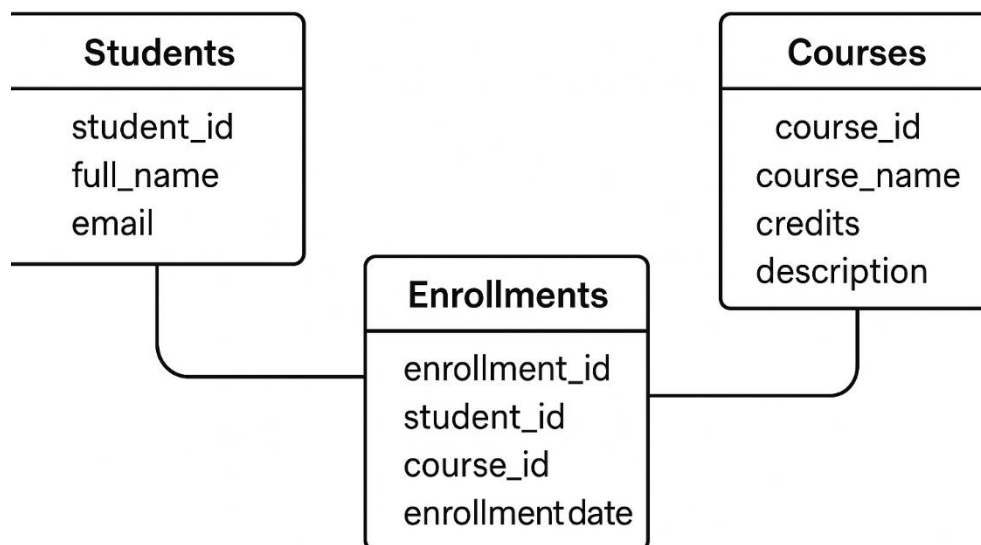
7. Delete (Drop) a Table

```
DROP TABLE Enrollments;
```

8. Export ER Diagram (DB-Specific Command)

MySQL Workbench:

Database → Reverse Engineer → Export as PNG/PDF



MODEL VIVA QUESTIONS

1. What is DDL in SQL?
2. Name different DDL commands
3. What is the CHECK constraint used for?
4. What are indexes and why are they used?
5. How do you add a new column to an existing table?

RESULT

The query has been executed successfully and the result is obtained.

EXP NO:3**DATA INSERTION AND BULK IMPORT USING DML COMMANDS AND UI TOOLS****AIM**

To understand how to insert data into database tables, perform bulk data import using UI and SQL commands, and verify successful data initialization using SQL queries.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Perform data insertion using SQL INSERT commands.
2. Import large datasets using bulk import tools or LOAD DATA SQL command.
3. Understand the purpose of data initialization in database implementation.
4. Validate inserted data using SELECT queries.
5. Solve real-time problems related to data import and integrity.

DESCRIPTION

Database initialization refers to loading initial data into tables after creating the database schema.

This includes:

1. Data Insert – adding individual rows using SQL INSERT commands.
2. Bulk Import – uploading large datasets quickly using DBMS UI tools (like MySQL Workbench, pgAdmin) or SQL commands such as LOAD DATA INFILE or COPY.
3. Verification – checking whether data is correctly inserted using SELECT queries.

This process is essential before starting operations like searching, updating, or analyzing data.

SQL QUERY QUESTIONS

1. Insert a new course into the Courses table.
2. Insert multiple students into the Students table.

3. Insert enrollment records by referencing foreign keys.
4. Update a student's email ID.
5. Delete a specific course.
6. Display all students enrolled in a specific course.
7. Count the total number of students.
8. Show all courses with more than 3 credits.
9. Perform bulk import of data into the Students table.
10. Verify imported data using a SELECT query.

SQL QUERY

To insert data to tables used in experiment no 2 using insert commands.

1. Insert a new course into the Courses table.

```
INSERT INTO Courses (course_id, course_name, credits, description)
```

```
VALUES
```

```
(101, 'Database Systems', 4, 'Relational DBMS concept'),
```

```
(102, 'Computer Networks', 3, 'Network fundamentals'),
```

```
(103, 'Python Programming', 4, 'Programming basics and OOP');
```

```
SELECT * FROM Courses;
```

course_id	course_name	credits	description
101	Database Systems	4	Relational DBMS concept
102	Computer Networks	3	Network fundamentals
103	Python Programming	4	Programming basics

2. Insert multiple students into the Students table.

```
INSERT INTO Students (student_id, full_name, email, dob)
```

```
VALUES
```

```
(1, 'Rahul Kumar', 'rahul@gmail.com', '2002-05-10'),
```

```
(2, 'Aiswarya KP', 'aiswarya@gmail.com', '2001-12-20'),  
(3, 'John Mathew', 'john@gmail.com', '2000-08-15');  
SELECT * FROM Student;
```

3. Insert enrollment records by referencing foreign keys.

```
INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date)  
VALUES  
(501, 1, 101, '2024-01-10'),  
(502, 2, 101, '2024-01-12'),  
(503, 3, 103, '2024-01-15');
```

4. SELECT * FROM Enrollments;

enrollment_id	student_id	course_id	enrollment_date
501	1	101	2024-01-10
502	2	101	2024-01-12
503	3	103	2024-01-15

5. Update a student's email ID.

```
UPDATE Students  
SET email = 'rahul_new@gmail.com'  
WHERE student_id = 1;  
SELECT * FROM Students WHERE student_id = 1;
```

student_id	full_name	email	dob
1	Rahul Kumar	rahul_new@gmail.com	2002-05-10

6. Delete a specific course.

```
DELETE FROM Courses  
WHERE course_id = 102;  
  
SELECT * FROM Courses;
```

course_id	course_name	credits	description
101	DBMS	4	Intro to DBMS
103	Computer Networks	3	Networking fundamentals

7. Display all students enrolled in a specific course.

```
SELECT s.student_id, s.full_name, s.email, e.course_id
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

student_id	full_name	email	course_id
1	Rahul Kumar	rahul_new@gmail.com ↗	101
2	Aiswarya KP	aiswarya@gmail.com ↗	101

8. Count the total number of students.

```
SELECT COUNT(*) FROM Students;
```

```
COUNT(*)
```

```
103
```

Show all courses with more than 3 credits.

```
SELECT * FROM Courses
WHERE credits > 3;
```

course_id	course_name	credits	description
101	DBMS	4	Intro to DBMS
104	Data Science	5	Basics of Data Sci

9. Perform bulk import of data into the Students table.

Create a file named students.csv

Add the data in the following format (with header row):

```
student_id,full_name,email,dob
3,Anu Nair,anu@gmail.com,2001-08-14
4,Manu R,manu@gmail.com,2002-01-22
5,Riya Das,riya@gmail.com,2003-11-05
```

10. SQL Bulk Import Command:

```
LOAD DATA INFILE '/path/students.csv'
INTO TABLE Students
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
Verify imported data using a SELECT query.
SELECT * FROM Students;
```

student_id	full_name	email	dob
1	Rahul Kumar	rahul_new@gmail.com ↗	2002-05-10
2	Aiswarya KP	aiswarya@gmail.com ↗	2003-06-18
3	Anu Nair	anu@gmail.com ↗	2001-08-14
4	Manu R	manu@gmail.com ↗	2002-01-22
5	Riya Das	riya@gmail.com ↗	2003-11-05

MODEL VIVA QUESTIONS

1. What is bulk import?
2. Why do we use INSERT INTO command?
3. What happens if a foreign key value doesn't exist in the parent table?
4. What is the difference between INSERT and BULK IMPORT?
5. What is the purpose of data initialization?

RESULT

The database was successfully initialized by inserting individual records, performing bulk data import, and verifying the inserted data. Students learned how to use SQL commands for data insertion, manage large dataset imports, and validate data integrity using SQL queries.

EXP NO:4**SQL DML: ESSENTIAL DATA MANIPULATION OPERATIONS****AIM:**

To practice and execute SQL DML (Data Manipulation Language) commands such as insertion, updating, altering, deletion of data, and querying records based on specific conditions in a database.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

- Understand and apply INSERT, UPDATE, DELETE, and SELECT queries.
- Modify existing data correctly and safely inside database tables.
- Apply conditions using WHERE, LIKE, BETWEEN, and comparison operators.
- Retrieve filtered and meaningful information from a database.
- Understand the role of DML commands in manipulating real-time data.
- Distinguish between DDL and DML operations.

DESCRIPTION

Data Manipulation Language (DML) commands are used to add, modify, delete, and retrieve data from database tables. After the database schema is defined using DDL commands, DML commands help manage and operate on the actual data stored in the tables.

1. INSERT Command

Used to add new rows of data into a table.

Syntax: INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

2. UPDATE Command

Used to modify existing data in a table.

Syntax: UPDATE table_name SET column_name = value WHERE condition;

3. DELETE Command

Used to remove specific rows from a table.

Syntax: DELETE FROM table_name WHERE condition;

4. SELECT Command

Used to retrieve and display data from one or more tables.

Syntax: SELECT column1, column2, ... FROM table_name WHERE condition;

Additional Options:

- Sorting: ORDER BY
- Filtering: WHERE, LIKE, BETWEEN
- Joining tables: JOIN

These DML operations are essential for maintaining accurate and up-to-date information in any database. They form the core of daily data activities performed in systems such as banking, e-commerce, college management, and hospital databases.

By practicing DML commands, students gain a clear understanding of how data is inserted, updated, deleted, validated, and retrieved in real-world applications.

SQL QUERY QUESTIONS

Consider the employee database created in Find results for the following questions

1. Find the names of all employees who work for SBI.
2. Find all employees in the database who live in the same cities as the companies for which they work.
3. Find all employees and their managers in the database who live in the same cities and on the same street number as do their managers.
4. Find all employees who earn more than the average salary of all employees of their company.
5. Find the company that pay least total salary along with the salary paid.
6. Give all managers of SBI a 10 percent raise.
7. Find the company that has the most employees

8. Find those companies whose employees earn a higher salary, on average than the average salary at Indian Bank.
9. Query to find name and salary of all employees who earn more than each employee of 'Indian Bank'

SQL QUERY

Employees Table

emp_id	emp_name	street	city	salary	company_name	manager_id
1	Arjun	MG Road	Kochi	50000	SBI	3
2	Neha	Anna St	Chennai	45000	Indian Bank	3
3	Manoj	MG Road	Kochi	70000	SBI	NULL
4	Riya	MG Road	Kochi	30000	HDFC	3
5	Kiran	Anna St	Chennai	28000	Indian Bank	2
6	Meera	MG Road	Kochi	55000	SBI	3

Company Table

company_name	city
SBI	Kochi
Indian Bank	Chennai
HDFC	Kochi

1. Find the names of all employees who work for SBI.

SELECT emp_name FROM Employees WHERE company_name = 'SBI';

emp_name

Arjun

Manoj

Meera

2. Find all employees in the database who live in the same cities as the companies for which they work.

```
SELECT e.emp_name, e.city, e.company_name FROM Employees e
JOIN Company c ON e.company_name = c.company_name WHERE e.city = c.city;
```

emp_name	city	company_name
Arjun	Kochi	SBI
Manoj	Kochi	SBI
Meera	Kochi	SBI
Neha	Chennai	Indian Bank
Kiran	Chennai	Indian Bank

- 3. Find all employees and their managers in the database who live in the same cities and on the same street number as do their managers.**

```
SELECT e.emp_name, m.emp_name AS manager_name
FROM Employees e
JOIN Employees m ON e.manager_id = m.emp_id
WHERE e.city = m.city AND e.street = m.street;
```

emp_name	manager_name
Arjun	Manoj
Riya	Manoj
Meera	Manoj

- 4. Find all employees who earn more than the average salary of all employees of their company.**

```
SELECT e.emp_name, e.salary, e.company_name FROM Employees e
WHERE e.salary > ( SELECT AVG(salary)
FROM Employees WHERE company_name = e.company_name);
```

emp_name	salary	company_name
Manoj	70000	SBI
Neha	45000	Indian Bank

5. Find the company that pays the least total salary along with the salary paid.

```
SELECT company_name, SUM(salary) AS total_salary FROM Employees
GROUP BY company_name ORDER BY total_salary ASC
LIMIT 1;
```

company_name	total_salary
HDFC	30000

6. Give all managers of SBI a 10 percent raise.

```
UPDATE Employees SET salary = salary * 1.10 WHERE emp_id IN (
SELECT DISTINCT manager_id FROM Employees WHERE company_name = 'SBI');
```

emp_id	emp_name	salary
3	Manoj	77000

7. Find the company that has the most employees

```
SELECT company_name, COUNT(*) AS total_employees
FROM Employees
GROUP BY company_name
ORDER BY total_employees DESC
LIMIT 1;
```

company_name	total_employees
SBI	3

8. Find those companies whose employees earn a higher salary, on average than the average salary at Indian Bank.

```
SELECT company_name FROM Employees GROUP BY company_name
HAVING AVG(salary) > (SELECT AVG(salary) FROM Employees
WHERE company_name = 'Indian Bank');
```

company_name

SBI

9. Query to find name and salary of all employees who earn more than each employee of 'Indian Bank'

```
SELECT emp_name, salary FROM Employees WHERE salary > ALL (SELECT salary  
FROM Employees WHERE company_name = 'Indian Bank' );
```

emp_name	salary
Manoj	70000
Meera	55000

MODEL VIVA QUESTIONS

1. What is the difference between DDL and DML?
2. Write any two DML commands.
3. What is the purpose of the WHERE clause?
4. Can we update multiple rows at the same time?
5. What happens if you delete a row without a WHERE clause?

RESULT:

The student successfully practiced various SQL DML commands including insertion, updation, deletion, altering data, and retrieving records based on conditions. The experiment helped in understanding how data is stored, modified, and queried in a relational database.

EXP NO:5**IMPLEMENTATION OF SQL AGGREGATE FUNCTIONS****AIM:**

To implement and understand various SQL aggregate functions (SUM, AVG, MAX, MIN, COUNT) and use ORDER BY, GROUP BY, and HAVING clauses to organize, summarize, and filter grouped database records.

LEARNING OUTCOMES

By the end of this experiment, students will be able to:

1. Apply aggregate functions to summarize data in SQL tables.
2. Use GROUP BY to group records based on specific columns.
3. Use HAVING to filter grouped results.
4. Sort records using ORDER BY in ascending/descending order.
5. Write queries that combine aggregation, grouping, filtering, and sorting effectively.

DESCRIPTION**1. Aggregate Functions**

Aggregate functions perform calculations on a set of values and return a single value.

Function	Description
SUM(column)	Returns total sum
AVG(column)	Returns average value
MAX(column)	Returns maximum value
MIN(column)	Returns minimum value
COUNT(column)	Returns number of records

Syntax SELECT AGGREGATE_FUNCTION(column) FROM table_name;

2. GROUP BY Clause

Used to group rows that have the same values in specified columns.

Syntax: SELECT column, AGGREGATE_FUNCTION(column) FROM table_name

GROUP BY column;

3. HAVING Clause

Used to filter groups after applying the GROUP BY clause.

Syntax: SELECT column, AGGREGATE_FUNCTION(column) FROM table_name GROUP BY column HAVING condition;

4. ORDER BY Clause

Used to sort the result set in ascending (ASC) or descending (DESC) order.

Syntax: SELECT column1, column2 FROM table_name ORDER BY column1 ASC|DESC;

SQL QUERY QUESTIONS

Create the tables with the following fields

Faculty (FacultyCode, FacultyName)

Subject (SubjectCode, SubjectName, MaxMark, FacultyCode)

Student(StudentCode, StudentName, DOB, StudentsBranch(CS/EC/EE/ME),
AdmissionDate)

M_Mark (StudentCode, SubjectCode, Mark)

1. Display the number of faculties.
2. Display the total mark for each student.
3. Display the subject, average mark for each subject.
4. Display the name of subjects for which at least one student got below 40%.
5. Display the name, subject and percentage of marks who got below 40 %.
6. Display the faculties and allotted subjects for each faculty
7. Display the name of faculties who take more than one subject.
8. Display name, subject, mark, % of mark in ascending order of mark

SQL COMMANDS:

1. Display the number of faculties.

```
SELECT COUNT(*) AS Number_of_Faculties FROM Faculty;
```

Number_of_Faculties
3

2. Display the total mark for each student.

```
SELECT s.StudentCode, s.StudentName, SUM(m.Mark) AS TotalMarks FROM Student s
```

```
JOIN M_Mark m ON s.StudentCode = m.StudentCode
```

```
GROUP BY s.StudentCode, s.StudentName;
```

StudentCode	StudentName	TotalMarks
S01	Arjun	245
S02	Meera	260
S03	Rahul	230

3. Display the subject, average mark for each subject.

```
SELECT sub.SubjectName, AVG(m.Mark) AS AverageMark
```

```
FROM Subject sub
```

```
JOIN M_Mark m ON sub.SubjectCode = m.SubjectCode
```

```
GROUP BY sub.SubjectName;
```

SubjectName	AverageMark
DBMS	75.5
Networks	68.0
Mathematics	82.3

4. Display the name of subjects for which at least one student got below 40%.

```
SELECT DISTINCT sub.SubjectName FROM Subject sub

JOIN M_Mark m ON sub.SubjectCode = m.SubjectCode

WHERE m.Mark < (0.40 * sub.MaxMark);
```

SubjectName

Networks

DBMS

5. Display the name, subject and percentage of marks who got below 40 %.

```
SELECT s.StudentName, sub.SubjectName, (m.Mark / sub.MaxMark) * 100 AS Percentage

FROM Student s JOIN M_Mark m ON s.StudentCode = m.StudentCode

JOIN Subject sub ON m.SubjectCode = sub.SubjectCode

WHERE m.Mark < (0.40 * sub.MaxMark);
```

StudentName	SubjectName	Percentage
Rahul	Networks	35%
Arjun	DBMS	38%

6. Display the faculties and allotted subjects for each faculty.

```
SELECT f.FacultyName, s.SubjectName

FROM Faculty f LEFT JOIN Subject s ON f.FacultyCode = s.FacultyCode;
```

FacultyName	SubjectName
Dr. Ramesh	DBMS
Dr. Ramesh	Networks
Ms. Anju	Mathematics
Mr. Sajan	NULL

7. **Display the name of faculties who take more than one subject.**

```
SELECT f.FacultyName, COUNT(s.SubjectCode) AS No_of_Subjects
FROM Faculty f JOIN Subject s ON f.FacultyCode = s.FacultyCode
GROUP BY f.FacultyName HAVING COUNT(s.SubjectCode) > 1;
```

FacultyName	No_of_Subjects
Dr. Ramesh	2

8. **Display name, subject, mark, % of mark in ascending order of mark**

```
SELECT s.StudentName, sub.SubjectName, m.Mark,
(m.Mark / sub.MaxMark) * 100 AS Percentage FROM Student s JOIN
M_Mark m ON s.StudentCode = m.StudentCode JOIN Subject sub ON m.SubjectCode =
sub.SubjectCode ORDER BY m.Mark ASC;
```

StudentName	SubjectName	Mark	Percentage
Arun	Math	35	35%
Neha	DBMS	55	55%
Arun	DBMS	78	78%
Rahul	Physics	90	90%

MODEL VIVA QUESTIONS

1. What is the difference between WHERE and HAVING?
2. Why is GROUP BY used in SQL?
3. Name any four aggregate functions with examples.
4. Can we use aggregate functions without GROUP BY? Explain.
5. What is the use of ORDER BY clause?

RESULT:

The experiment was successfully performed. Aggregate functions were applied, and the use of ORDER BY, GROUP BY, and HAVING clauses was demonstrated effectively.

EXP NO: 6**IMPLEMENTATION OF SET OPERATORS, NESTED QUERIES, AND JOIN QUERIES****AIM**

To practice and implement SQL queries using set operators (UNION, INTERSECT, MINUS), nested queries, and join queries to retrieve data from multiple tables effectively.

LEARNING OUTCOMES

After performing this experiment, students will be able to:

1. Understand and implement SQL set operators (UNION, INTERSECT, MINUS) to manipulate datasets.
2. Write and execute nested queries (subqueries) for complex data retrieval.
3. Understand and apply different join operations (INNER JOIN, LEFT JOIN, RIGHT JOIN) to combine data from multiple tables.
4. Analyze query results and understand the relationship between tables in a relational database.
5. Enhance SQL skills for real-world database querying and data manipulation.

DESCRIPTION

Set Operators :-Set operators are used to combine the results of two or more SELECT statements.

Both queries must have the same number of columns with compatible data types.

UNION: Combines results of two queries and removes duplicate rows. Returns only distinct records from both result sets.

Syntax:

```
SELECT column1 FROM table1
```

```
UNION
```

```
SELECT column1 FROM table2;
```

UNION ALL: Same as UNION but retains duplicate rows (no duplicate elimination), making it faster.

INTERSECT: Returns only the rows that are common to both queries.

Syntax:

```
SELECT column1 FROM table1
```

INTERSECT

```
SELECT column1 FROM table2;
```

MINUS (also called EXCEPT in some databases): Returns rows from the first query that do not appear in the second query.

Syntax:

```
SELECT column1 FROM table1
```

MINUS

```
SELECT column1 FROM table2;
```

Nested Queries (Subqueries)

A nested query, or subquery, is a query written inside another query. The inner query executes first and its result is used by the outer query. Subqueries can appear in the WHERE, FROM, or SELECT clause.

Single-row subquery: Returns only one row, used with operators like =, <, >.

```
SELECT name FROM employee
```

```
WHERE salary > (SELECT AVG(salary) FROM employee);
```

Multi-row subquery: Returns multiple rows, used with operators like IN, ANY, ALL.

```
SELECT name FROM employee
```

```
WHERE dept_id IN (SELECT dept_id FROM department WHERE location = 'Chennai');
```

Correlated subquery: The inner query depends on the outer query and is executed once for each row processed by the outer query.

Subqueries are useful for filtering, comparing aggregated values, and performing complex conditional retrievals that would be difficult with a single query.

Join Queries

Joins are used to combine rows from two or more tables based on a related column between them.

INNER JOIN: Returns only the matching rows present in both tables.

```
SELECT e.name, d.dept_name
```

```
FROM employee e
```

```
INNER JOIN department d ON e.dept_id = d.dept_id;
```

LEFT JOIN (LEFT OUTER JOIN): Returns all rows from the left table and matching rows from the right table; unmatched right-table columns show NULL.

RIGHT JOIN (RIGHT OUTER JOIN): Returns all rows from the right table and matching rows from the left table; unmatched left-table columns show NULL.

FULL OUTER JOIN: Returns all rows when there is a match in either table, with NULL where no match exists.

SELF JOIN: A table is joined with itself, useful for hierarchical or comparative data within the same table.

SQL QUERY QUESTIONS

1. Create two tables Employee1 and Employee2 with fields: emp_id, emp_name, salary.
2. Write SQL queries using set operators to:
 - a. Find all employees from both tables (UNION).
 - b. Find common employees in both tables (INTERSECT).
 - c. Find employees in Employee1 but not in Employee2 (MINUS).
3. Create a table Department with fields: dept_id, dept_name.
4. Create a table EmployeeDept with fields: emp_id, dept_id.
5. Write nested queries to:
 - a. List employees working in a specific department.
 - b. Find employees with salaries greater than the average salary.
6. Write join queries to:
 - a. Display employee details along with their department names using INNER JOIN.
 - b. Display all departments with employees (including departments with no employees) using LEFT JOIN.

SQL QUERY

1. **Create two tables Employee1 and Employee2 with fields: emp_id, emp_name, salary.**

```
CREATE TABLE Employee1 ( emp_id INT PRIMARY KEY, emp_name VARCHAR(20),  
salary DECIMAL(10,2));
```

```
CREATE TABLE Employee2 (emp_id INT PRIMARY KEY, emp_name VARCHAR(20),  
salary DECIMAL(10,2));
```

```
INSERT INTO Employee1 VALUES (1,'Alice',45000);
```

```
INSERT INTO Employee1 VALUES (2,'Bob',50000);
```

```
INSERT INTO Employee1 VALUES (3,'Charlie',47000);
```

```
INSERT INTO Employee2 VALUES (2,'Bob',50000);
```

```
INSERT INTO Employee2 VALUES (3,'Charlie',47000);
```

```
SELECT * FROM Employee1;
```

```
SELECT * FROM Employee2;
```

```
mysql> CREATE TABLE Employee1 (
-> emp_id INT PRIMARY KEY,
-> emp_name VARCHAR(20),
-> salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Employee2 (
-> emp_id INT PRIMARY KEY,
-> emp_name VARCHAR(20),
-> salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO Employee1 VALUES (1, 'Alice', 45000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee1 VALUES (2, 'Bob', 50000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee2 VALUES (2, 'Bob', 50000);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee2 VALUES (3, 'Charlie', 47000);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM Employee1;
+-----+-----+-----+
| emp_id | emp_name | salary |
+-----+-----+-----+
| 1      | Alice   | 45000.00 |
| 2      | Bob     | 50000.00 |
| 3      | Charlie | 47000.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

2. Write SQL queries using set operators to:

- Find all employees from both tables (UNION).

```
SELECT emp_name FROM Employee1 UNION SELECT emp_name FROM Employee2;
```

```
mysql> SELECT emp_name FROM Employee1
-> UNION
-> SELECT emp_name FROM Employee2;
+-----+
| emp_name |
+-----+
| Alice    |
| Bob      |
| Charlie  |
+-----+
3 rows in set (0.00 sec)

mysql>
```

- b. Find common employees in both tables (INTERSECT).

```
SELECT emp_name FROM Employee1 INTERSECT SELECT emp_name FROM Employee2;
```

```
mysql> SELECT emp_name FROM Employee1
-> INTERSECT
-> SELECT emp_name FROM Employee2;
+-----+
| emp_name |
+-----+
| Bob      |
| Charlie  |
+-----+
2 rows in set (0.00 sec)

mysql>
```

- c. Find employees in Employee1 but not in Employee2 (MINUS).

```
SELECT emp_name FROM Employee1 MINUS SELECT emp_name FROM Employee2;
```

```
mysql> SELECT emp_name FROM Employee1
-> MINUS
-> SELECT emp_name FROM Employee2;
+-----+
| emp_name |
+-----+
| Alice     |
+-----+
1 row in set (0.00 sec)

mysql>
```

3. Create a table Department with fields: dept_id, dept_name.

```
CREATE TABLE Department (dept_id INT PRIMARY KEY, dept_name VARCHAR(20));
INSERT INTO Department VALUES (101,'HR');
INSERT INTO Department VALUES (102,'IT');
INSERT INTO Department VALUES (103,'Finance');
```

```
mysql> SELECT * FROM Department;
+-----+-----+
| dept_id | dept_name |
+-----+-----+
| 101     | HR        |
| 102     | IT        |
| 103     | Finance   |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

4. Create a table EmployeeDept with fields: emp_id, dept_id.

```
CREATE TABLE EmployeeDept (emp_id INT,dept_id INT,
FOREIGN KEY (emp_id) REFERENCES Employee1(emp_id),
FOREIGN KEY (dept_id) REFERENCES Department(dept_id));
INSERT INTO EmployeeDept VALUES (1,101);
INSERT INTO EmployeeDept VALUES (2,102);
INSERT INTO EmployeeDept VALUES (3,102);
```

```
mysql> CREATE TABLE EmployeeDept (
-> emp_id INT,
-> dept_id INT,
-> FOREIGN KEY (emp_id) REFERENCES Employee1(emp_id),
-> FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO EmployeeDept VALUES (1,101);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EmployeeDept VALUES (2,102);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EmployeeDept VALUES (3,102);
Query OK, 1 row affected (0.00 sec)
```

5. Write nested queries to:

- a. List employees working in a specific department.

```
SELECT emp_name FROM Employee1 WHERE emp_id IN ( SELECT emp_id
FROM EmployeeDept WHERE dept_id = 102);
```

```
mysql> SELECT emp_name
-> FROM Employee1
-> WHERE emp_id IN (
-> SELECT emp_id
-> FROM EmployeeDept
-> WHERE dept_id = 102
-> );
+-----+
| emp_name |
+-----+
| Anjali   |
| Kiran    |
| David    |
+-----+
3 rows in set (0.01 sec)
```

- b. Find employees with salaries greater than the average salary.

SELECT emp_name, salary FROM Employee1

WHERE salary > (SELECT AVG(salary) FROM Employee1);

```
mysql> SELECT emp_name, salary
-> FROM Employee1
-> WHERE salary > (SELECT AVG(salary) FROM Employee1);
+-----+-----+
| emp_name | salary |
+-----+-----+
| Rahul    | 65000  |
| Anjali   | 72000  |
| Kiran    | 68000  |
+-----+-----+
3 rows in set (0.01 sec)
```

6. Write join queries to:

- a. Display employee details along with their department names using INNER JOIN.

SELECT e.emp_name, d.dept_name FROM EmployeeDept ed

INNER JOIN Employee1 e ON ed.emp_id = e.emp_id

INNER JOIN Department d ON ed.dept_id = d.dept_id;

```
mysql> SELECT e.emp_name, d.dept_name
-> FROM EmployeeDept ed
-> INNER JOIN Employee1 e ON ed.emp_id = e.emp_id
-> INNER JOIN Department d ON ed.dept_id = d.dept_id;
+-----+-----+
| emp_name | dept_name |
+-----+-----+
| Rahul    | HR        |
| Anjali   | Finance   |
| Kiran    | IT        |
| David    | Marketing |
+-----+-----+
4 rows in set (0.01 sec)
```

- b. **Display all departments with employees (including departments with no employees) using LEFT JOIN.**

SELECT d.dept_name, e.emp_name FROM Department d LEFT JOIN EmployeeDept ed

ON d.dept_id = ed.dept_id LEFT JOIN Employee1 e ON ed.emp_id = e.emp_id;

```
mysql> SELECT d.dept_name, e.emp_name
-> FROM Department d
-> LEFT JOIN EmployeeDept ed ON d.dept_id = ed.dept_id
-> LEFT JOIN Employee1 e ON ed.emp_id = e.emp_id;

+-----+-----+
| dept_name | emp_name |
+-----+-----+
| HR        | Rahul    |
| Finance   | Anjali   |
| IT        | Kiran    |
| Marketing | David    |
| Admin     | NULL     |
+-----+-----+
5 rows in set (0.01 sec)
```

MODEL VIVA QUESTIONS

1. What is a set operator in SQL? Name the types of set operators.
2. Can you update or delete data using a view?
3. What is the difference between MINUS (or EXCEPT) and INTERSECT?
4. When would you use a subquery instead of a join?
5. What happens if the tables in a join have no matching records?

RESULT

All SQL queries executed successfully using set operators, nested queries, and joins. The outputs accurately show unions, intersections, differences, and employee–department relationships.

EXP NO:7 A.

IMPLEMENTATION OF SQL TCL COMMANDS

AIM

To practice and understand Transaction Control Language (TCL) commands such as COMMIT, ROLLBACK, and SAVEPOINT, which help in managing transactions and maintaining data integrity in a database.

LEARNING OUTCOMES

After performing this experiment, students will be able to:

1. Understand the concept of transactions in SQL.
2. Use COMMIT to permanently save changes made to a database.
3. Use ROLLBACK to undo unwanted changes.
4. Create and use SAVEPOINTS to partially roll back a transaction.

Ensure data consistency and error recovery using TCL commands.

DESCRIPTION

TRANSACTIONAL CONTROL LANGUAGE (T.C.L): A transaction is a logical unit of work. All changes made to the database can be referred to as a transaction. Transaction changes can be made permanent to the database only if they are committed; a transaction begins with an executable SQL statement & ends explicitly with either roll back or commit statement.

COMMIT: This command is used to end a transaction only with the help of the commit command transaction changes can be made permanent to the database.

Syntax: SQL>COMMIT;

Example: SQL>COMMIT;

SAVE POINT: Save points are like marks to divide a very lengthy transaction to smaller once. They are used to identify a point in a transaction to which we can later roll back. Thus, save points are used in conjunction with roll back.

Syntax: SQL>SAVE POINT ID;

Example: SQL>SAVE POINT xyz;

ROLL BACK: A roll back command is used to undo the current transactions. We can roll back the entire transaction so that all changes made by SQL statements are undo (or) roll back a transaction to a save point so that the SQL statements after the save point are rolled back.

Syntax:

ROLL BACK(current transaction can be roll back)

ROLL BACK to save point ID;

Example:

SQL>ROLL BACK;

SQL>ROLL BACK TO SAVE POINT xyz;

SAVE POINT: Save points are like marks to divide a very lengthy transaction to smaller once. They are used to identify a point in a transaction to which we can later roll back. Thus, save point is used in conjunction with roll back.

Syntax:

SQL> SAVE POINT ID;

Example: SQL> SAVE POINT xyz;

SQL QUERY QUESTIONS

1. Create a database NEW.
2. Create a table CLASS with the ID and Name of the student.
3. Insert 5 rows into the table.
4. Display the table.
5. Begin transaction and insert a new row into the table.
6. Create a savepoint.
7. Insert a new row into the table and display the table.
8. Rollback to savepoint and display the table.
9. Update the table with the new name of the student having ID=5 and display the table.
10. Rollback to savepoint and display the table.
11. Finally commit the transaction.

SQL COMMANDS:

1. Create a database NEW.

```
CREATE DATABASE NEW;
```

```
USE NEW;
```

```
mysql> CREATE DATABASE NEW;
Query OK, 1 row affected (0.01 sec)

mysql> USE NEW;
Database changed

mysql>
```

2. Create a table CLASS with the ID and Name of the student.

```
CREATE TABLE CLASS (ID INT PRIMARY KEY, Name VARCHAR(50));
```

```
mysql> CREATE TABLE CLASS (
  -> ID INT PRIMARY KEY,
  -> Name VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
```

3. Insert 5 rows into the table.

```
INSERT INTO CLASS VALUES (1, 'Avani');
```

```
INSERT INTO CLASS VALUES (2, 'Sree');
```

```
INSERT INTO CLASS VALUES (3, 'Indra');
```

```
INSERT INTO CLASS VALUES (4, 'Nanda');
```

```
INSERT INTO CLASS VALUES (5, 'Bhoomi');
```

```
mysql> INSERT INTO CLASS VALUES (1, 'Avani');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (2, 'Sree');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (3, 'Indra');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (4, 'Nanda');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (5, 'Bhoomi');
Query OK, 1 row affected (0.00 sec)
```

4. Display the table.

```
SELECT * FROM CLASS;
```

5. Begin transaction and insert a new row into the table.

START TRANSACTION;

INSERT INTO CLASS VALUES (6, 'Krishna');

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (6, 'Krishna');
Query OK, 1 row affected (0.00 sec)
```

6. Create a savepoint.

SAVEPOINT SP1;

7. Insert a new row into the table and display the table.

INSERT INTO CLASS VALUES (7, 'Shiva');

SELECT * FROM CLASS;

```
mysql> INSERT INTO CLASS VALUES (7, 'Shiva');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM CLASS;
+----+-----+
| ID | Name  |
+----+-----+
| 1  | Avani |
| 2  | Sree  |
| 3  | Indra  |
| 4  | Nanda |
| 5  | Bhoomi|
| 6  | Krishna|
| 7  | Shiva |
+----+-----+
7 rows in set (0.00 sec)
```

8. Rollback to savepoint and display the table.

ROLLBACK TO SP1;

SELECT * FROM CLASS;

```
mysql> ROLLBACK TO SP1;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM CLASS;
+-----+-----+
| ID | Name |
+-----+-----+
| 1 | Avani |
| 2 | Sree |
| 3 | Indra |
| 4 | Nanda |
| 5 | Bhoomi |
| 6 | Krishna |
+-----+-----+
6 rows in set (0.00 sec)

mysql>
```

9. Update the table with the new name of the student having ID=5 and display the table.

UPDATE CLASS SET Name = 'Shiva' WHERE ID = 5;

SELECT * FROM CLASS;

```
mysql> UPDATE CLASS SET Name = 'Shiva' WHERE ID = 5;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM CLASS;
+-----+-----+
| ID | Name |
+-----+-----+
| 1 | Avani |
| 2 | Sree |
| 3 | Indra |
| 4 | Nanda |
| 5 | Shiva |
| 6 | Krishna |
+-----+-----+
6 rows in set (0.00 sec)
```

10. Rollback to savepoint and display the table.

ROLLBACK TO SP1;

SELECT * FROM CLASS;

```
mysql> ROLLBACK TO SP1;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM CLASS;
+-----+
| ID | Name |
+-----+
| 1 | Avani |
| 2 | Sree |
| 3 | Indra |
| 4 | Nanda |
| 5 | Bhoomi |
| 6 | Krishna |
+-----+
6 rows in set (0.00 sec)

mysql>
```

11. Finally commit the transaction.

COMMIT;

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

RESULT:

TCL commands such as COMMIT, ROLLBACK, and SAVEPOINT were successfully executed.

EXP NO:7 B**IMPLEMENTATION OF SQL DCL COMMANDS****AIM:**

To practice SQL Data Control Language (DCL) commands such as GRANT and REVOKE used to manage user privileges in a database.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand database security and access control.
2. Grant specific privileges to users on database objects.
3. Revoke assigned privileges as needed.
4. Differentiate between system and object privileges.
5. Implement controlled data access in a multi-user environment.

DESCRIPTION:

PRIVILEGES: A privilege is a right to execute an SQL statement or to access another user's object. In Oracle, there are two types of privileges

- System Privileges
- Object Privileges

System Privileges are those through which the user can manage the performance of database actions. It is normally granted by DBA to users. Eg: Create Session, Create Table, Create user etc..

Object Privileges allow access to objects or privileges on objects, i.e. tables, table columns, tables, views etc.. It includes alter, delete, insert, select update etc. (After creating the user, DBA grant specific system privileges to user)

GRANT: The DBA uses the GRANT statement to allocate system privileges to other users.

Syntax:

SQL> GRANT privilege [privilege....] TO USER ;

SQL> Grant succeeded

Eg: Grant create session, create table, create view to James;

Object privileges vary from object to object.

An owner has all privileges or specific privileges on objects.

SQL> GRANT object_priv [(column)] ON object TO user;

SQL>GRANT select, insert ON emp TO James;

SQL>GRANT select ,update (e_name,e_address) ON emp TO James;

CHANGE PASSWORD: The DBA creates an account and initializes a password for every user.You can change password by using ALTER USER statement.

Syntax:

Alter USER IDENTIFIED BY

REVOKE: A REVOKE statement is used to remove privileges granted to other users.The privileges you specify are revoked from the users.

Syntax:

REVOKE [privilege.. ...] ON object FROM user

Eg:

REVOKE create session,create table from James;

REVOKE select ,insert ON emp FROM James

ROLE:A role is a named group of related privileges that can be granted to a user.In other words, role is a predefined collection of privileges that are grouped together,thus privileges are easier to assign users.

SQL> Create role custom;

SQL> Grant create table, create view TO custom;

SQL> Grant select, insert ON emp TO custom;

SQL QUERY QUESTIONS

1. Create a new user USER1 with a password.
2. Grant CREATE, SELECT and INSERT privileges to the new user, USER1.

3. Exit from the root user.
4. Login to the new user, USER1.
5. Insert a new row into the CLASS table of database NEW.
6. Display the table CLASS and Exit from USER1.
7. Login to the root user and REVOKE INSERT privileges of USER1 from the CLASS table of database NEW.
8. Exit from the root user.
9. Login to the new user, USER1 and insert a new row into the CLASS table of database NEW.

SQL COMMANDS:

1. Create a new user USER1 with a password.

```
CREATE USER 'USER1'@'localhost' IDENTIFIED BY 'password123';
```

```
CREATE DATABASE NEW;
```

```
USE NEW;
```

```
CREATE TABLE CLASS (ID INT PRIMARY KEY, Name VARCHAR(50));
```

```
INSERT INTO CLASS VALUES (1, 'Avani');
```

```
INSERT INTO CLASS VALUES (2, 'Sree');
```

```
INSERT INTO CLASS VALUES (3, 'Indra');
```

```
INSERT INTO CLASS VALUES (4, 'Nanda');
```

```
INSERT INTO CLASS VALUES (5, 'Bhoomi');
```

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi

2. Grant CREATE, SELECT and INSERT privileges to the new user, USER1.

```
GRANT CREATE, SELECT, INSERT ON *.* TO 'USER1'@'localhost';
```

```
FLUSH PRIVILEGES;
```

3. Exit from the root user.

```
EXIT;
```

4. Login to the new user, USER1.

```
mysql -u USER1 -p
```

5. Insert a new row into the CLASS table of database NEW.

```
USE NEW;
```

```
INSERT INTO CLASS (ID, Name) VALUES (6, 'Sujith');
```

6. Display the table CLASS and Exit from USER1.

```
SELECT * FROM CLASS;
```

ID	Name
1	Avani
2	Sree
3	Indra
4	Nanda
5	Bhoomi
6	Sujith

```
EXIT;
```

- 7. Login to the root user and REVOKE INSERT privileges of USER1 from the CLASS table of database NEW.**

```
REVOKE INSERT ON NEW.CLASS FROM 'USER1'@'localhost';
```

- 8. Exit from the root user.**

```
EXIT;
```

- 9. Login to the new user, USER1 and insert a new row into the CLASS table of database NEW.**

Login back in as USER1.

```
INSERT INTO CLASS (ID, Name) VALUES (2, 'Science');
```

MODEL VIVA QUESTIONS

1. What is the difference between DCL and TCL?
2. Give examples of DCL commands.
3. What is the use of the GRANT command?
4. What happens when privileges are revoked?
5. What is the difference between system and object privileges?

RESULT:

DCL commands GRANT and REVOKE were successfully executed.

EXP NO:8**CREATION AND MANIPULATION OF TABLES, VIEWS, AND
ASSERTIONS IN SQL****AIM**

To practice SQL commands for the creation and management of **Views** and **Assertions** in a database system.

LEARNING OUTCOMES

After completing this practice, students will be able to:

1. Create Views: Understand how to create virtual tables (views) using SQL commands to simplify complex queries.
2. Manipulate Views: Learn how to select, update, and delete data using views.
3. Understand Assertions: Learn how to create assertions to enforce integrity constraints that cannot be captured by traditional primary and foreign key constraints.
4. Apply SQL: Effectively use SQL to manage data integrity and query optimization.

DESCRIPTION

Views (Virtual Tables) - A View is a virtual table whose content is defined by a stored SQL query. It does not store data but rather pulls current data from one or more underlying base tables every time it is accessed.

Syntax:

```
CREATE VIEW ViewName AS SELECT column1, column2, ... FROM TableName(s)
```

WHERE condition;

Syntax for Deletion:

```
DROP VIEW ViewName;
```

Assertions (Database Constraints): An Assertion defines a condition that the database must always maintain across its state. It is a powerful form of integrity constraint that can involve multiple tables or complex calculations..

Syntax:

CREATE ASSERTION AssertionName CHECK (Condition);

SQL QUERY QUESTIONS

1. Create two tables STUDENT_DETAILS and ACCOUNT_DETAILS with the following columns:

STUDENT_DETAILS:

- std_id
- std_name
- branch

ACCOUNT_DETAILS:

- std_id
- std_name
- sem_fee

2. Create a view VIEW1 that contains std_id and branch.
3. Create a view VIEW2 that contains std_id, std_name, sem_fee, and branch.
4. Perform UPDATE and DELETE operations in the created views.
5. Insert a new row into VIEW1.
6. Drop the view.
7. Create an assertion that enforces the rule that std_id in STUDENT_DETAILS is always NOT NULL, since each student must have one std_id.

SQL COMMANDS:**1. Table 1 STUDENT_DETAILS**

CREATE TABLE STUDENT_DETAILS (std_id INT PRIMARY KEY,
std_name VARCHAR(100) NOT NULL, branch VARCHAR(50));

Table 2: ACCOUNT_DETAILS

```
CREATE TABLE ACCOUNT_DETAILS (  
std_id INT PRIMARY KEY,  
sem_fee DECIMAL(10, 2),  
FOREIGN KEY (std_id) REFERENCES STUDENT_DETAILS(std_id));  
  
INSERT INTO STUDENT_DETAILS (std_id, std_name, branch) VALUES  
(1, 'Alice', 'CSE'),  
(2, 'Bob', 'ECE'),  
(3, 'Charlie', 'CSE');  
  
INSERT INTO ACCOUNT_DETAILS (std_id, sem_fee) VALUES  
(1, 45000.00),  
(2, 50000.00),  
(3, 45000.00);
```

2. Create a view VIEW1 that contains std_id and branch.

```
CREATE VIEW view1 AS  
SELECT std_id, branch  
FROM student_details;
```

```
SELECT * FROM VIEW1;
```

std_id	branch
1	CSE
2	ECE
3	CSE

3. Create a view VIEW2 that contains std_id, std_name, sem_fee, and branch.

```
CREATE VIEW view2 AS  
SELECT s.std_id, s.std_name, a.sem_fee, s.branch  
FROM student_details s  
JOIN account_details a  
ON s.std_id = a.std_id;
```

```
SELECT * FROM VIEW2;
```

std_id	std_name	sem_fee	branch
1	Alice	45000.00	CSE
2	Bob	50000.00	ECE
3	Charlie	45000.00	CSE

4. Perform UPDATE and DELETE operations in the created views.

```
UPDATE view2
```

```
SET sem_fee = sem_fee + 1000
```

```
WHERE std_id = 1;
```

std_id	std_name	sem_fee	branch
1	Alice	46000.00	CSE
2	Bob	50000.00	ECE
3	Charlie	45000.00	CSE

```
DELETE FROM view2 WHERE std_id = 2;
```

std_id	std_name	sem_fee	branch
1	Alice	46000.00	CSE
3	Charlie	45000.00	CSE

5. Insert a new row into VIEW1.

```
INSERT INTO view1 VALUES (4, 'MECH');
```

std_id	branch
1	CSE
2	ECE
3	CSE
4	MECH

6. Drop the view.

```
DROP VIEW view1;
```

```
DROP VIEW view2;
```

7. **Create an assertion that enforces the rule that std_id in STUDENT_DETAILS is always NOT NULL, since each student must have one std_id.**

```
ALTER TABLE STUDENT_DETAILS
```

```
MODIFY std_id NOT NULL;
```

```
CREATE ASSERTION std_id_not_null CHECK ( NOT EXISTS (  
SELECT * FROM STUDENT_DETAILS WHERE std_id IS NULL));
```

MODEL VIVA QUESTIONS

1. What is a View in SQL, and how is it different from a table?
2. Can you update data through a View? If yes, under what conditions?
3. How would you enforce a condition like "age cannot be less than 18" for every row in a table?
4. What is the purpose of an assertion in SQL?
5. What is the difference between a VIEW and a MATERIALIZED VIEW

RESULT:

The SQL queries were executed successfully, and the required outputs for table creation, views, view manipulation, and assertion were obtained.

EXP NO:9 A**CREATION AND EXECUTION OF STORED PROCEDURES****AIM**

To study and implement Stored Procedures in SQL/PL-SQL for performing modular and reusable database operations.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand the concept and purpose of stored procedures.
2. Create and execute procedures in SQL/PL/SQL.
3. Pass IN, OUT, and INOUT parameters to procedures.
4. Perform DML operations using procedures.
5. Improve performance with precompiled SQL blocks.

DESCRIPTION

A Stored Procedure is a precompiled block of SQL statements stored in the database. It is executed on demand and can accept parameters.

Advantages

- Improves performance
- Reduces network traffic
- Increases reusability
- Provides security

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name (parameter1 datatype, parameter2  
datatype) IS BEGIN
```

```
-- SQL statements
```

```
END;
```

ALGORITHM

1. Start
2. Create a table (e.g., STUDENT).
3. Write a procedure to insert a new record.
4. Compile the procedure.
5. Execute the procedure using EXEC or an anonymous block.
6. Display the updated table.
7. End.

SQL QUERY QUESTIONS

1. Create the required tables (Employee, Emp_Log).
2. Write and compile the Stored Procedure to insert data.
3. Execute the procedure to insert sample employee records.

ALGORITHM

1. Define the stored procedure using CREATE PROCEDURE.
2. Specify input/output parameters (if any).
3. Write the SQL logic within the procedure.
4. Use the CALL or EXEC statement to execute the procedure.

SQL COMMANDS

Table creation:

```
CREATE TABLE STUDENT (  
  
    id NUMBER PRIMARY KEY,  
  
    name VARCHAR2(50),  
  
    marks NUMBER );
```

Procedure

```
CREATE OR REPLACE PROCEDURE add_student(  
  
    p_id IN NUMBER,  
  
    p_name IN VARCHAR2,
```

```
p_marks IN NUMBER)
```

```
IS
```

```
BEGIN
```

```
    INSERT INTO STUDENT VALUES (p_id, p_name, p_marks);
```

```
END;
```

```
/
```

Execution

```
EXEC add_student(1, 'Aiswarya', 92);
```

```
EXEC add_student(2, 'Meera', 88);
```

EXPECTED OUTPUT

```
Procedure created.
```

```
PL/SQL procedure successfully completed.
```

```
SELECT * FROM STUDENT;
```

ID	NAME	MARKS
1	Aiswarya	92
2	Meera	88

MODEL VIVA QUESTIONS

1. What is a stored procedure?
2. Why do we use stored procedures?
3. What is the difference between a function and a procedure?
4. Can procedures return values?
5. What are IN, OUT, and INOUT parameters?

RESULT

A stored procedure was successfully created and executed for performing insertion operations in a database.

EXP NO:9 B**CREATION AND IMPLEMENTATION OF DATABASE TRIGGERS****AIM**

To create and implement row-level and statement-level triggers in SQL/PL-SQL to automate database actions.

LEARNING OUTCOMES

Students will be able to:

1. Understand the purpose of triggers.
2. Create BEFORE and AFTER triggers.
3. Automatically maintain logs using triggers.
4. Understand row-level vs statement-level triggers.
5. Implement integrity checks using triggers.

DESCRIPTION

A Trigger is a stored PL/SQL block that executes automatically when an event occurs in a table.

Types of Triggers

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER DELETE
- ROW LEVEL
- STATEMENT LEVEL

Syntax

CREATE OR REPLACE TRIGGER trigger_name

BEFORE INSERT ON table_name

FOR EACH ROW

BEGIN

-- Actions

END;

/

ALGORITHM

- Start
- Create a main table and a log table.
- Create a trigger that fires whenever a record is inserted.
- Insert a record into the main table.
- Trigger automatically enters a log record.
- Display both tables.
- End.

SQL COMMANDS

Main Table

```
CREATE TABLE EMPLOYEE (  
  
    emp_id NUMBER PRIMARY KEY,  
  
    name VARCHAR2(50),  
  
    salary NUMBER  
  
);
```

Log Table

```
CREATE TABLE EMP_LOG (  
  
    emp_id NUMBER,  
  
    time_stamp DATE,  
  
    action VARCHAR2(20)  
  
);
```

Trigger

```
CREATE OR REPLACE TRIGGER log_employee_insert  
  
AFTER INSERT ON EMPLOYEE  
  
FOR EACH ROW  
  
BEGIN  
  
    INSERT INTO EMP_LOG VALUES(:NEW.emp_id, SYSDATE, 'INSERT');  
  
END;  
  
/
```

Execution

```
INSERT INTO EMPLOYEE VALUES (101, 'Akhil', 35000);
```

```
INSERT INTO EMPLOYEE VALUES (102, 'Diya', 38000);
```

EXPECTED OUTPUT

EMPLOYEE Table:

emp_id	name	salary
101	Akhil	35000
102	Diya	38000

EMP_LOG Table:

emp_id	time_stamp	action
101	24-NOV-2025 10:42AM	INSERT
102	24-NOV-2025 10:43AM	INSERT

MODEL VIVA QUESTIONS

1. What is a trigger?
2. What events can cause a trigger to fire?
3. What is the difference between BEFORE and AFTER triggers?
4. What is the purpose of the :NEW and :OLD keywords?

5. Can a trigger call another trigger?

RESULT:

Triggers were successfully created and executed, and automatic logging of insert operations was demonstrated.

EXP NO:9 C**CREATION AND EXECUTION OF USER-DEFINED FUNCTIONS****AIM**

To write and execute user-defined functions in PL/SQL for performing computations and returning values.

LEARNING OUTCOMES

Students will be able to:

1. Create functions using PL/SQL.
2. Understand return types.
3. Pass parameters to a function.
4. Use functions inside SQL queries.
5. Perform modular computations.

DESCRIPTION

A Function is a stored PL/SQL subprogram that must return a value using a RETURN statement.

Syntax:

```
CREATE OR REPLACE FUNCTION function_name(param datatype)
```

```
RETURN return_datatype
```

```
IS
```

```
BEGIN
```

```
-- Calculation
```

```
RETURN value;
```

```
END;
```

```
/
```

ALGORITHM

1. Start
2. Create a function to compute bonus from salary.

3. Return computed value.
4. Call the function inside a SELECT statement.
5. Display the output.
6. End.

SQL COMMANDS

Function to Calculate Bonus = 10% of Salary

Function

```
CREATE OR REPLACE FUNCTION calc_bonus(
```

```
    p_salary NUMBER
```

```
) RETURN NUMBER
```

```
IS
```

```
BEGIN
```

```
    RETURN p_salary * 0.10;
```

```
END;
```

```
/
```

Execution

```
SELECT emp_id, name, salary, calc_bonus(salary) AS bonus
```

```
FROM EMPLOYEE;
```

EXPECTED OUTPUT

emp_id	name	salary	bonus
101	Akhil	35000	3500
102	Diya	38000	3800

MODEL VIVA QUESTIONS

1. What is a function in PL/SQL?
2. How is a procedure different from a function?
3. Can a function return multiple values?
4. Can a function be used inside a SELECT query?

5. What is the purpose of the RETURN keyword?

RESULT:

A PL/SQL function was successfully created and executed, and the returned values were used in SQL queries.

EXP NO:10 A**CREATION AND IMPLEMENTATION OF PL/SQL PACKAGES****AIM**

To study and implement explicit cursors in PL/SQL to retrieve and process query results row by row.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Understand implicit and explicit cursors.
2. Declare and use an explicit cursor in PL/SQL.
3. Perform OPEN–FETCH–CLOSE cursor operations.
4. Process multiple rows using loops and cursor attributes.
5. Use cursor attributes such as %NOTFOUND, %FOUND, %ROWCOUNT.

DESCRIPTION

A cursor is a pointer to the result set of a SQL query. It allows row-by-row processing of database records.

Types:

1. Implicit Cursor – automatically created by Oracle for single-row operations.
2. Explicit Cursor – declared by programmer for multi-row queries.

Cursor operations:

1. DECLARE
2. OPEN
3. FETCH
4. CLOSE

Explicit Cursor Syntax

DECLARE

CURSOR cursor_name IS

SELECT column_list FROM table_name;

BEGIN

OPEN cursor_name;

FETCH cursor_name INTO variables;

CLOSE cursor_name;

END;

/

ALGORITHM

1. Start
2. Create a table (STUDENT) and insert sample records
3. Declare variables to store fetched values
4. Declare an explicit cursor with SELECT statement
5. OPEN the cursor
6. FETCH rows inside a loop
7. EXIT when %NOTFOUND is TRUE
8. CLOSE the cursor
9. Display the output using DBMS_OUTPUT
10. End

SQL COMMANDS:

Create Table

```
CREATE TABLE STUDENT (  
    id NUMBER PRIMARY KEY,  
    name VARCHAR2(50),  
    marks NUMBER  
);
```

```
INSERT INTO STUDENT VALUES (1, 'Aiswarya', 92);
```

```
INSERT INTO STUDENT VALUES (2, 'Meera', 85);
```

```
INSERT INTO STUDENT VALUES (3, 'Arun', 78);
```

Explicit Cursor Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
CURSOR stu_cursor IS
    SELECT id, name, marks FROM STUDENT;
v_id STUDENT.id%TYPE;
v_name STUDENT.name%TYPE;
v_marks STUDENT.marks%TYPE;
BEGIN
    OPEN stu_cursor;
    LOOP
        FETCH stu_cursor INTO v_id, v_name, v_marks;
        EXIT WHEN stu_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_marks);
    END LOOP;
    CLOSE stu_cursor;
END;
/
```

EXPECTED OUTPUT

```
1 || Aiswarya || 92
2 || Meera    || 85
3 || Arun     || 78
```

MODEL VIVA QUESTIONS

1. What is a cursor?
2. Difference between implicit and explicit cursor?
3. What are cursor attributes?
4. Why do we need OPEN, FETCH, CLOSE?
5. Can we use multiple cursors in PL/SQL blocks?

RESULT:

The cursor was successfully created and executed, and records were fetched and displayed row by row

EXP NO:10 B**IMPLEMENTATION OF CURSORS IN PL/SQL****AIM**

To study, create, and execute PL/SQL packages containing procedures, functions, and declarations for modular programming.

LEARNING OUTCOMES

Students will be able to:

1. Understand the structure of packages (Specification & Body).
2. Create and execute procedures and functions inside a package.
3. Use package variables and cursors.
4. Understand advantages of packages in PL/SQL.
5. Develop modular and reusable database components.

DESCRIPTION

A Package in PL/SQL is a collection of related program units stored together.

Packages contain:

- Procedures
- Functions
- Variables
- Cursors

Packages consist of two parts:

1. Package Specification

Defines the interface (public elements).

2. Package Body

Contains the actual code (implementation).

Benefits:

- Modularity
- Better performance

- Security
- Reusability

Syntax of Package Specification

```
CREATE OR REPLACE PACKAGE package_name AS

    PROCEDURE procedure_name(parameters);

    FUNCTION function_name(parameters) RETURN datatype;

END package_name;

/
```

Syntax of Package Body

```
CREATE OR REPLACE PACKAGE BODY package_name AS

    PROCEDURE procedure_name IS
    BEGIN
        -- code
    END;

    FUNCTION function_name RETURN datatype IS
    BEGIN
        RETURN value;
    END;

END package_name;

/
```

ALGORITHM

1. Start
2. Create table STUDENT
3. Create package specification declaring procedure and function
4. Create package body implementing procedure and function
5. Execute procedure and function
6. Display results
7. End

SQL COMMANDS:

```
CREATE TABLE STUDENT ( id NUMBER PRIMARY KEY, name VARCHAR2(50), marks
NUMBER);
```

Package Specification

```
INSERT INTO STUDENT VALUES (1, 'Aiswarya', 92);
INSERT INTO STUDENT VALUES (2, 'Meera', 85);
CREATE OR REPLACE PACKAGE student_pack AS
    PROCEDURE add_student(p_id NUMBER, p_name VARCHAR2, p_marks NUMBER);
    FUNCTION get_marks(p_id NUMBER) RETURN NUMBER;
END student_pack;
/
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY student_pack AS
    PROCEDURE add_student(p_id NUMBER, p_name VARCHAR2, p_marks NUMBER) IS
    BEGIN
        INSERT INTO STUDENT VALUES (p_id, p_name, p_marks);
    END;
    FUNCTION get_marks(p_id NUMBER) RETURN NUMBER IS
        v_marks NUMBER;
    BEGIN
        SELECT marks INTO v_marks FROM STUDENT WHERE id = p_id;
        RETURN v_marks;
    END;
END student_pack;
/
```

Execute the Package

```
EXEC student_pack.add_student(3, 'Arun', 78);
SELECT student_pack.get_marks(1) AS Marks FROM dual;
```

EXPECTED OUTPUT

sql

1 row inserted.

MARKS

92

MODEL VIVA QUESTIONS

1. What is a package in PL/SQL?
2. Difference between package specification and package body.
3. Advantages of using packages.
4. Can a package contain a cursor?
5. What happens if the package body is missing?

RESULT:

Packages were successfully created and executed. Procedures and functions within the package worked correctly.

EXP NO:11**DATABASE APPLICATION DESIGN USING A FRONT-END TOOL****AIM**

To design and develop a database application using a front-end tool (such as VB, Java, Python, PHP, .NET, or HTML-CSS-JavaScript) integrated with a relational database system, based on the problem selected in Experiment 1, using a minimum of five tables.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

- Understand how to integrate a front-end interface with a back-end database.
- Design multiple database tables with relationships.
- Apply SQL queries for data manipulation through a GUI.
- Develop CRUD operations (Create, Read, Update, Delete) using forms.
- Build a functional mini-application demonstrating real-world data flow.

DESCRIPTION

A database application is software that interacts with a database to store, retrieve, update, and manage data.

It consists of:

1. Front-End

- The user interface of the application.
- Created using tools like Python Tkinter, Java Swing, JSP, HTML/PHP, .NET, etc.
- Performs actions like:
 - Form submission
 - Displaying reports
 - User input validation

2. Back-End

- The database where actual data is stored.
- Managed using DBMS such as MySQL, Oracle, PostgreSQL, SQL Server.

- Includes:
 - Tables
 - Views
 - Procedures
 - Triggers
 - Access controls

3. Application Workflow

1. The user interacts with the front-end UI.
2. The application sends SQL commands to the database.
3. The database processes and returns results.
4. The application displays results to the user.

ALGORITHM

- 1) Start the program.
- 2) Import all required modules:
 - tkinter for GUI
 - mysql.connector for database connection
- 3) Create a MySQL connection using hostname, username, password, and database name.
- 4) Create the main Tkinter window with labels, text fields, and buttons.
- 5) Define Functions for:
 - Adding a student record
 - Viewing all records
 - Updating a student record
 - Deleting a student record
- 6) In the Add function:
 - Collect data from text fields
 - Insert into the Student table
 - Commit the transaction
- 7) In the View function:
 - Execute SELECT query
 - Display all results in listbox
- 8) In the Update function:
 - Collect student ID
 - Update specific fields using UPDATE query

- Commit the transaction
- 9) In the Delete function:
- Collect student ID
 - Execute DELETE query
 - Commit the transaction
- 10) Bind each button with its respective function.
- 11) Run the Tkinter main loop to display GUI.
- 12) Stop the program after closing the window.

SQL COMMANDS:

```
import mysql.connector

from tkinter import *

from tkinter import messagebox

from tkinter import ttk

# -----

# DATABASE CONNECTION

# -----

def connect_db():

    try:

        conn = mysql.connector.connect(

            host="localhost",

            user="root",

            password="", # add your MySQL password

            database="college_app"

        )

        return conn

    except:
```

```
messagebox.showerror("Error", "Database Connection Failed")
```

```
return None
```

```
# -----
```

```
# INSERT STUDENT
```

```
# -----
```

```
def insert_student():
```

```
    sid = entry_sid.get()
```

```
    name = entry_name.get()
```

```
    dob = entry_dob.get()
```

```
    branch = entry_branch.get()
```

```
    conn = connect_db()
```

```
    cur = conn.cursor()
```

```
    try:
```

```
        cur.execute("INSERT INTO Student VALUES (%s, %s, %s, %s)",
```

```
                    (sid, name, dob, branch))
```

```
        conn.commit()
```

```
        messagebox.showinfo("Success", "Student Added Successfully")
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Failed: {e}")
```

```
    conn.close()
```

```
# -----
```

```
# VIEW STUDENTS
```

```
# -----
```

```
def view_students():
```

```
    conn = connect_db()
```

```
    cur = conn.cursor()
```

```
    cur.execute("SELECT * FROM Student")
```

```
    rows = cur.fetchall()
```

```
    tree.delete(*tree.get_children())
```

```
    for row in rows:
```

```
        tree.insert("", END, values=row)
```

```
    conn.close()
```

```
# UPDATE FEES STATUS
```

```
def update_fees():
```

```
    fee_id = entry_feeid.get()
```

```
    status = entry_status.get()
```

```
    conn = connect_db()
```

```
    cur = conn.cursor()
```

```
    try:
```

```
        cur.execute("UPDATE Fees SET status=%s WHERE fee_id=%s", (status, fee_id))
```

```
        conn.commit()
```

```
        messagebox.showinfo("Success", "Fees updated successfully")
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Failed: {e}")
```

```
    conn.close()
```

DELETE STAFF

```
def delete_staff():

    staff = entry_staff.get()

    conn = connect_db()

    cur = conn.cursor()

    try:

        cur.execute("DELETE FROM Staff WHERE staff_id=%s", (staff,))

        conn.commit()

        messagebox.showinfo("Success", "Staff Deleted Successfully")

    except Exception as e:

        messagebox.showerror("Error", f"Failed: {e}")

    conn.close()
```

SEARCH STUDENT

```
def search_student():

    name = entry_search.get()

    conn = connect_db()

    cur = conn.cursor()

    cur.execute("SELECT * FROM Student WHERE stud_name LIKE %s", ("%"+name+"%",))

    rows = cur.fetchall()

    tree.delete(*tree.get_children())

    for row in rows:

        tree.insert("", END, values=row)

    conn.close()
```

GUI DESIGN

```
root = Tk()

root.title("College Database Application")

root.geometry("900x600")

title = Label(root, text="COLLEGE DATABASE APPLICATION", font=("Arial", 18, "bold"))

title.pack()

frame1 = LabelFrame(root, text="Insert Student", padx=10, pady=10)

frame1.place(x=20, y=60)

Label(frame1, text="Student ID").grid(row=0, column=0)

Label(frame1, text="Name").grid(row=1, column=0)

Label(frame1, text="DOB (YYYY-MM-DD)").grid(row=2, column=0)

Label(frame1, text="Branch").grid(row=3, column=0)

entry_sid = Entry(frame1)

entry_sid.grid(row=0, column=1)

entry_name = Entry(frame1)

entry_name.grid(row=1, column=1)

entry_dob = Entry(frame1)

entry_dob.grid(row=2, column=1)

entry_branch = Entry(frame1)

entry_branch.grid(row=3, column=1)

Button(frame1, text="Add Student", command=insert_student).grid(row=4, column=0,
columnspan=2)

# ----- FEES UPDATE -----

frame2 = LabelFrame(root, text="Update Fees Status", padx=10, pady=10)

frame2.place(x=20, y=250)
```

```
Label(frame2, text="Fee ID").grid(row=0, column=0)

Label(frame2, text="Status").grid(row=1, column=0)

entry_feeid = Entry(frame2)

entry_feeid.grid(row=0, column=1)

entry_status = Entry(frame2)

entry_status.grid(row=1, column=1)

Button(frame2, text="Update Fees", command=update_fees).grid(row=2, column=0, columnspan=2)

# ----- STAFF DELETE -----

frame3 = LabelFrame(root, text="Delete Staff", padx=10, pady=10)

frame3.place(x=20, y=400)

Label(frame3, text="Staff ID").grid(row=0, column=0)

entry_staff = Entry(frame3)

entry_staff.grid(row=0, column=1)

Button(frame3, text="Delete Staff", command=delete_staff).grid(row=1, column=0, columnspan=2)

# ----- SEARCH STUDENT -----

frame4 = LabelFrame(root, text="Search Student", padx=10, pady=10)

frame4.place(x=350, y=60)

Label(frame4, text="Enter Name").grid(row=0, column=0)

entry_search = Entry(frame4)

entry_search.grid(row=0, column=1)

Button(frame4, text="Search", command=search_student).grid(row=1, column=0, columnspan=2)

# ----- TREE VIEW (TABLE DISPLAY) -----

tree = ttk.Treeview(root, columns=("ID", "Name", "DOB", "Branch"), show="headings")

tree.place(x=350, y=150, width=520, height=350)
```

```
tree.heading("ID", text="Student ID")

tree.heading("Name", text="Name")

tree.heading("DOB", text="DOB")

tree.heading("Branch", text="Branch")

Button(root, text="View All Students", command=view_students).place(x=350, y=520)

root.mainloop()
```

Create database:

```
CREATE DATABASE college_app;
```

```
USE college_app;
```

Table 1: STUDENT

```
CREATE TABLE Student (

    stud_id INT PRIMARY KEY,

    stud_name VARCHAR(100),

    dob DATE,

    branch VARCHAR(50)

);
```

Table 2: COURSE

```
CREATE TABLE Course (

    course_id INT PRIMARY KEY,

    course_name VARCHAR(100),

    credits INT

);
```

Table 3 Enrollment

```
CREATE TABLE Enrollment (
```

```
enroll_id INT PRIMARY KEY,  
  
stud_id INT,  
  
course_id INT,  
  
sem INT,  
  
FOREIGN KEY (stud_id) REFERENCES Student(stud_id),  
  
FOREIGN KEY (course_id) REFERENCES Course(course_id)  
);
```

Table 4 Fees

```
CREATE TABLE Fees (  
  
    fee_id INT PRIMARY KEY,  
  
    stud_id INT,  
  
    amount DECIMAL(10,2),  
  
    status VARCHAR(20),  
  
    FOREIGN KEY (stud_id) REFERENCES Student(stud_id)  
);
```

Table 5 Staff

```
CREATE TABLE Staff (  
  
    staff_id INT PRIMARY KEY,  
  
    staff_name VARCHAR(100),  
  
    designation VARCHAR(50)  
);
```

EXPECTED OUTPUT

1. Main GUI Window

pgsql

COLLEGE DATABASE APPLICATION	
INSERT STUDENT	SEARCH STUDENT
UPDATE FEES	(Student Table View Area)
DELETE STAFF	

2. After Adding a Student

Success: Student Added Successfully

3. View All Students Output in Table

ID	NAME	DOB	BRANCH
101	Aiswarya K P	2002-03-12	CSE
102	Meera N	2001-07-05	ECE
103	Arun P	2003-11-22	ME

4. Search Student Output

sql

Showing only "Aiswarya" in the table

5. Fees Update Message

Success: Fees updated successfully

6. Staff Delete Message

Success: Staff Deleted Successfully

PROCEDURE

1. Install Python (version 3.8 or above).

2. Install MySQL Server and create a database
3. Create table inside the database
4. Install required Python MySQL package by typing in terminal:
 - pip install mysql-connector-python
5. Create a Python file
6. Type the complete Python Tkinter code inside the file.
7. Save the file.
8. Open Command Prompt / Terminal in the folder where the file is stored.
9. Run the program
10. The GUI window will open.
11. Perform operations:
12. Enter details → Click ADD
13. Click VIEW → Display all data
14. Enter ID + new data → Click UPDATE
15. Enter ID → Click DELETE
16. Close the window to stop execution.

MODEL VIVA QUESTIONS

1. What is the purpose of using Tkinter in this project?
2. How is Python connected to MySQL in this project?
3. What are the different operations performed in your application?
4. Why do we use commit() after executing SQL commands?
5. What is the role of the mainloop() function in Tkinter?

RESULT:

A front-end GUI database application was successfully developed using Python Tkinter integrated with MySQL. The application supports inserting student details, viewing records, updating fees, deleting staff, and searching students. The GUI interacts with the database using SQL queries, and the expected outputs were successfully obtained.

EXP NO:12**CRUD OPERATIONS ON CASSANDRA DATABASE****AIM**

To understand and implement CRUD (Create, Read, Update, Delete) operations on a Cassandra database using CQL.

LEARNING OUTCOMES

- Understand the fundamental concepts of Cassandra database.
- Learn how to perform basic CRUD operations on Cassandra tables.
- Gain practical knowledge on interacting with Cassandra using CQL (Cassandra Query Language).
- Learn how to set up a Cassandra instance and connect to it programmatically.

DESCRIPTION

Apache Cassandra is an open-source, distributed NoSQL database system designed to handle large amounts of data across many commodity servers. It is highly scalable, fault-tolerant, and decentralized. Cassandra is often used in scenarios where high availability and scalability are critical, such as handling large-scale web applications or real-time analytics. Unlike relational databases, Cassandra does not use structured query language (SQL) for querying but instead uses Cassandra Query Language (CQL), which is very similar to SQL but designed specifically for Cassandra's NoSQL architecture.

Some key features of Cassandra include:

1. **Scalability:** Cassandra can easily scale horizontally by adding more nodes to the cluster without any downtime, making it suitable for applications with large data sets.
2. **Fault Tolerance:** It is designed to handle node failures gracefully, ensuring data availability even in the case of hardware failures.
3. **High Availability:** Cassandra's decentralized nature ensures that all nodes in a cluster are equal, meaning there is no single point of failure.
4. **Write-Optimized:** Cassandra is optimized for fast writes and is particularly useful for applications where the data is constantly being written and updated, such as log management or sensor data.

The CRUD operations (Create, Read, Update, Delete) are the fundamental actions that can be performed on data in Cassandra. These operations are typically executed using CQL queries:

CREATE

Insert new data into the database.

syntax

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

READ

Retrieve data from the database.

syntax

```
SELECT column1, column2 FROM table_name;
```

UPDATE

Modify existing data in the database.

syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE primary_key_column = value;
```

DELETE

Remove data from the database.

syntax

```
DELETE FROM table_name WHERE primary_key_column = value;
```

SQL QUERY QUESTIONS

1. Create a keyspace named collegeDB with a replication factor of 1 using SimpleStrategy.
2. In the keyspace collegeDB, create a table students with the following fields:
3. Insert at least 5 student records into the students table.
4. Write a query to display all student details from the table.
5. Write a query to retrieve details of students who belong to the Computer Science department.
6. Write a query to update the marks of a student with roll_no = 103.
7. Delete the record of the student whose roll_no = 105.
8. Alter the table and add a new column named email of type TEXT.
9. Insert 2 records in a single BATCH statement
10. Write a query to drop the table students from collegeDB.

SQL COMMANDS:

1. Create a keyspace named collegeDB with a replication factor of 1 using SimpleStrategy.

```
CREATE KEYSPACE collegedb  
WITH REPLICATION = {'class': 'SimpleStrategy', 'replication_factor': 1};
```

2. In the keyspace collegeDB, create a table students with the following fields:

```
USE collegedb;  
CREATE TABLE students (  
roll_no INT PRIMARY KEY,  
name TEXT,  
dept TEXT,  
marks INT  
);
```

3. Insert at least 5 student records into the students table.

```
INSERT INTO students (roll_no, name, dept, marks) VALUES (101, 'Arjun', 'Computer  
Science', 89);  
INSERT INTO students (roll_no,name,dept,marks) VALUES (102, 'Meera', 'Mechanical', 76);  
INSERT INTO students (roll_no,name,dept,marks) VALUES (103, 'Rahul', 'Computer  
Science',92);  
INSERT INTO students (roll_no, name,dept,marks) VALUES (104, 'Sneha', 'Electronics', 81);  
INSERT INTO students (roll_no, name, dept, marks) VALUES (105, 'Kiran', 'Civil', 72);
```

4. Write a query to display all student details from the table.

```
SELECT * FROM students;
```

roll_no	name	dept	marks
101	Arjun	Computer Science	89
102	Meera	Mechanical	76
103	Rahul	Computer Science	92
104	Sneha	Electronics	81
105	Kiran	Civil	72

5. Write a query to retrieve details of students who belong to the Computer Science department.

```
SELECT * FROM students  
WHERE dept = 'Computer Science' ALLOW FILTERING;
```

roll_no	name	dept	marks
101	Arjun	Computer Science	89
103	Rahul	Computer Science	92

6. Write a query to update the marks of a student with roll_no = 103.

```
UPDATE students SET marks = 95 WHERE roll_no = 103;
```

7. Delete the record of the student whose roll_no = 105.

```
DELETE FROM students WHERE roll_no = 105;
```

```
SELECT * FROM students
```

roll_no	name	dept	marks
101	Arjun	Computer Science	89
102	Meera	Mechanical	76
103	Rahul	Computer Science	95
104	Sneha	Electronics	81

8. Alter the table and add a new column named email of type TEXT.

```
ALTER TABLE students ADD email TEXT;
```

9. Insert 2 records in a single BATCH statement

```
BEGIN BATCH
```

```
INSERT INTO students (roll_no, name, dept, marks, email)
```

```
VALUES (201, 'Anita', 'ECE', 89, 'anita@mail.com');
```

```
INSERT INTO students (roll_no, name, dept, marks, email)
```

```
VALUES (202, 'Raghav', 'MECH', 76, 'raghav@mail.com');
```

```
APPLY BATCH;
```

```
SELECT * FROM students WHERE roll_no IN (201, 202);
```

roll_no	name	dept	marks	email
201	Anita	ECE	89	anita@mail.com
202	Raghav	MECH	76	raghav@mail.com

10. Write a query to drop the table students from collegeDB.

DROP TABLE students;

PROCEDURE

1. Install and start Apache Cassandra
2. Open terminal and type cqlsh
3. Execute keyspace and table creation commands
4. Insert records using INSERT
5. Retrieve records using SELECT
6. Update data using UPDATE
7. Delete records using DELETE
8. Verify results
9. Exit CQLSH

MODEL VIVA QUESTIONS

1. What is Cassandra?
2. What is a keyspace?
3. Define CRUD operations
4. What is a primary key?
5. Difference between SQL and CQL

RESULT

Successfully executed CRUD operations using Cassandra and CQL.

EXP NO:13**RETRIEVING SPECIFIC DATA FROM CASSANDRA USING CQL****AIM:**

To write and execute CQL (Cassandra Query Language) queries to retrieve specific records from Cassandra tables using SELECT commands with conditions, filtering, and column selection.

LEARNING OUTCOMES

After completing this experiment, the student will be able to:

1. Understand the structure of Cassandra tables
2. Use SELECT queries to retrieve data
3. Apply WHERE conditions for specific data retrieval
4. Use column-based selection
5. Understand limitations of filtering in Cassandra

DESCRIPTION

Apache Cassandra is a distributed NoSQL database designed for high availability and scalability.

To retrieve data from Cassandra, we use CQL (Cassandra Query Language), which is similar to SQL but optimized for distributed data storage.

Key Concepts:

- Keyspace → Database in Cassandra
- Table → Stores data in rows and columns
- Primary Key → Used to uniquely identify records
- SELECT Command → Used to retrieve data

Cassandra only allows data retrieval efficiently using:

- Primary Key
- Partition Key
- Indexed Columns

SQL QUERY QUESTIONS

1. Display all student records
2. Display a student with ID = 2
3. Display only name and department of students
4. Display students from "Computer Science" department
5. Display students whose age is greater than 20

SQL COMMANDS:

```
CREATE KEYSPACE college
```

```
WITH replication = {
```

```
  'class': 'SimpleStrategy',
```

```
  'replication_factor': 1
```

```
};
```

```
USE college;
```

```
CREATE TABLE students (
```

```
  id INT PRIMARY KEY,
```

```
  name TEXT,
```

```
  age INT,
```

```
  department TEXT
```

```
);
```

```
INSERT INTO students (id, name, age, department)
```

```
VALUES (1, 'Anu', 20, 'Computer Science');
```

```
INSERT INTO students (id, name, age, department)
```

```
VALUES (2, 'Rahul', 22, 'Electronics');
```

```
INSERT INTO students (id, name, age, department)
```

```
VALUES (3, 'Meera', 21, 'Mechanical');
```

```
INSERT INTO students (id, name, age, department)
```

```
VALUES (4, 'Akhil', 23, 'Computer Science');
```

1. Display all student records

```
SELECT * FROM students;
```

id	name	age	department
1	Anu	20	Computer Science
2	Rahul	22	Electronics
3	Meera	21	Mechanical
4	Akhil	23	Computer Science

2. Display a student with ID = 2

```
SELECT * FROM students WHERE id = 2;
```

id	name	age	department
2	Rahul	22	Electronics

3. Display only name and department of students

```
SELECT name, department FROM students;
```

name	department
Anu	Computer Science
Rahul	Electronics
Meera	Mechanical
Akhil	Computer Science

4. Display students from "Computer Science" department

```
SELECT * FROM students
```

```
WHERE department = 'Computer Science'
```

```
ALLOW FILTERING;
```

id	name	age	department
1	Anu	20	Computer Science
4	Akhil	23	Computer Science

5. Display students whose age is greater than 20

```
SELECT * FROM students
```

```
WHERE age > 20
```

```
ALLOW FILTERING;
```

id	name	age	department
2	Rahul	22	Electronics
3	Meera	21	Mechanical
4	Akhil	23	Computer Science

PROCEDURE

1. Open terminal
2. Start Cassandra server

```
cassandra
```

3. Open CQL shell

```
cqlsh
```

4. Create keyspace
5. Use keyspace
6. Create table
7. Insert records
8. Execute SELECT queries
9. Observe the outputs

MODEL VIVA QUESTIONS

1. What is CQL in Cassandra?
2. What is the difference between SQL and CQL?
3. What is a primary key in Cassandra?
4. Why is ALLOW FILTERING used?
5. Can we use WHERE without the primary key in Cassandra?

RESULT:

Thus, the CQL queries to retrieve specific data from Cassandra tables were successfully written and executed, and the output was verified.

EXP NO:14**SIMPLE CRUD APPLICATION USING MONGODB AND PYTHON****AIM**

To design and implement a simple CRUD (Create, Read, Update, Delete) application using Python as the programming language and MongoDB as the backend database to store and manage records efficiently.

LEARNING OUTCOMES

After completing this project, the student will be able to:

1. Understand the basics of NoSQL databases.
2. Connect Python applications with MongoDB.
3. Perform CRUD operations on a database.
4. Understand document-based data storage.
5. Develop simple backend logic using Python.
6. Compare SQL and NoSQL database concepts.

SOFTWARE REQUIREMENTS

- Python 3.x
- MongoDB
- PyMongo library
- Operating System: Windows / Linux

DESCRIPTION

CRUD Operations

CRUD stands for:

- Create – Insert new records into the database.
- Read – Retrieve records from the database.
- Update – Modify existing records.
- Delete – Remove records from the database.

MongoDB

MongoDB is a NoSQL, document-oriented database that stores data in JSON-like documents. It does not use tables and rows like SQL databases; instead, it uses collections and documents.

Python and MongoDB

Python connects to MongoDB using the PyMongo library, which provides methods for database operations.

ALGORITHM

1. Start the program.
2. Import the pymongo library.
3. Establish a connection with the MongoDB server.
4. Select database and collection.
5. Insert a record (Create).
6. Display records (Read).
7. Update a record (Update).
8. Delete a record (Delete).
9. Stop the program

CODE

```
from pymongo import MongoClient

# Connect to MongoDB

client = MongoClient("mongodb://localhost:27017/")

db = client["collegeDB"]

collection = db["students"]

# CREATE

collection.insert_one({"roll": 1, "name": "Alice", "marks": 85})

print("Record Inserted")

# READ

print("Records:")
```

```
for record in collection.find():
```

```
    print(record)
```

```
# UPDATE
```

```
collection.update_one({"roll": 1}, {"$set": {"marks": 90}})
```

```
print("Record Updated")
```

```
# DELETE
```

```
collection.delete_one({"roll": 1})
```

```
print("Record Deleted")
```

MONGODB QUERIES

```
db.students.insertOne({'roll': 1, 'name': 'Alice', 'marks': 85})
```

```
db.students.find()
```

```
db.students.updateOne({'roll': 1}, {'$set': {'marks': 90}})
```

```
db.students.deleteOne({'roll': 1})
```

OUTPUT

Record Inserted

Records:

```
{'_id': ObjectId('64f1a2c9e4b0c123456789ab'), 'roll': 1, 'name': 'Alice', 'marks': 85}
```

Record Updated

Record Deleted

PROCEDURE

1. Run MongoDB in command prompt or terminal
2. Install PyMongo library using: pip install pymongo
3. Create a Python file (example: mongodb_app.py) and write the program code.
4. Execute the program using: python mongodb_app.py
5. A student record is inserted into the students collection MongoDB automatically generates an _id for the record

6. All student records are retrieved using find()
7. The inserted record is displayed on the screen
8. The student's marks are updated from 88 to 92
9. The student record is deleted from the collection

MODEL VIVA QUESTIONS

1. What is CRUD?
2. What is MongoDB?
3. Difference between SQL and MongoDB?
4. What is a collection in MongoDB?
5. What is PyMongo?

RESULT

Thus, a simple CRUD application using MongoDB and Python was successfully designed and implemented. All CRUD operations were performed efficiently, demonstrating the functionality of a NoSQL database.